

**INDUSTRY INTERNSHIP
REPORT
ON**

" BANK MANAGEMENT SYSTEM "

SUBMITTED TO



J.C Bose University of Science and Technology, Faridabad

IN THE PARTIAL FULFILLMENT OF THE REQUIREMENT FOR THE AWARD OF THE DEGREE OF

BACHELOR OF TECHNOLOGY

IN

Computer Science and Engineering



Submitted To :
Ms. Rashika Singh
(Assistant Professor)

Submitted By :
Rajeev Singh
(21011004070)

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING
ARAVALI COLLEGE OF ENGINEERING AND MANAGEMENT, FARIDABAD-
121002

(June 2026)

TRAINING LETTER



Training Completion Certificate

This is to certify that

RAJEEV SINGH

has successfully completed 6 Months Training on

"JAVA Full Stack With DSA"

Mr. Anil Kumar Singh
Director

Started From
Monday, January 13, 2025

Mr. Vikas Kalra
Director



Verify this Certificate by Registration Number ETJAVA12820249M6500539 at <http://www.cetpainfotech.com>

CANDIDATE DECLARATION

I hereby declare that the project work entitled "Bank Management System" submitted to JC Bose University of Science and Technology Faridabad, Haryana (India), is a record of an original work done by me under the guidance of Ms. Rashika Singh (Assistant Professor) in Computer Science and Engineering, ARAVALI COLLEGE OF ENGINEERING AND MANAGEMENT, FARIDABAD, and this project work is submitted in the partial fulfillment of the requirements for the award of the degree of Bachelor of Technology in COMPUTER SCIENCE AND ENGINEERING.

Date

Student Name: Rajeev Singh

Roll Number : 21011004070

ACKNOWLEDGEMENT

This project would not have taken shape without the guidance provided by **Ms. Rashika Singh**, who helped in the modules of our project and resolved all the technical as well as other problems related to the project and, for always providing us with a helping hand whenever we faced any bottlenecks, in spite of being quite busy with their hectic schedules. Above all we wish to express our heartfelt gratitude to **Dr. Ashif Ali (H.O.D, CSE DEPARTMENT)** whose support has greatly boosted our self-confidence and will go a long way in helping us to reach further milestones and greater heights.

Rajeev Singh

TABLE OF CONTENTS

A. TRAINING LETTER	i
B. DECLARATION BY CANDIDATE.....	ii
C. ACKNOWLEDGEMENT.....	iii
D. COMPANY PROFILE.....	iv
<u>1. INTRODUCTION</u>	<u>01</u>
1.1 Project Summary.....	02
1.2 Project Objective.....	03
1.3 Tolls Hardware And Software Requirements.....	04
1.4 Project Category.....	05
<u>2. SYSTEM STUDY</u>	<u>06 - 08</u>
2.1 Preliminary Investigation.....	09
2.2 System Development.....	10
2.3 Feasibility Study.....	12
2.3.1 Technical Feasibility.....	13
2.3.2 Operational Feasibility.....	14
2.3.3 Economic Feasibility.....	15
<u>3. SYSTEM ANALYSIS</u>	<u>16 - 19</u>
3.1 Existing System.....	20
3.2 Proposed System.....	21
<u>4. SYSTEM DESIGN</u>	<u>22 - 25</u>
4.1 Entity Relationship Diagram.....	26 - 31
<u>5. SYSTEM DEVELOPMENT</u>	<u>32</u>
5.1 Coding.....	33 - 76

6. TESTING	77
6.1 Testing.....	78
6.2 Testing Cases.....	79 - 81
6.3 Verification and Validation.....	82
6.4 Debugging Feasibility.....	83
7. SCREENSHOTS	84
7.1 Login Page.....	85
7.2 Registration Page.....	85
7.3 Partner Landing Page.....	86
7.4 Services Offered.....	86
7.5 Accounts Balance.....	87
7.6 Record Page.....	87
8. IMPLEMENTATION OF THE SECURITY FOR THE SOFTWARE DEVELOPED	88 - 89
9. LIMITATION	90 - 91
10. CONCLUSION AND SCOPE OF FUTURE APPLICATION	92 - 96
REFERENCES	97
BRIEF PROFLIE	98

COMPANY PROFILE

CETPA Infotech Pvt. Ltd. is a leading ISO 9001:2015 certified organization that specializes in technical training, software development, and career consultancy services, offering a comprehensive ecosystem for skill development and industry engagement. Founded in 2002, CETPA has established itself as a technology-driven learning and development platform that bridges the gap between academic education and industry readiness. The company operates multiple training centers across India, with major hubs in Noida, Roorkee, Dehradun, and Lucknow, and has also extended its footprint to global markets through international collaborations.

CETPA's core competency lies in its structured and modular training programs designed to equip students and working professionals with practical, project-based learning across diverse technology domains. The curriculum spans over 50 high-demand fields, including programming languages, web and mobile app development, artificial intelligence, embedded systems, networking, cloud computing, data science, and digital marketing. These offerings are delivered via flexible modes—offline, online, and hybrid—to accommodate the evolving needs of learners and organizations.

The organization adopts a standardized training methodology that includes live project work, industry case studies, mock interviews, and certification exams, ensuring that learners not only acquire theoretical knowledge but also gain hands-on experience. CETPA's dedicated placement cell complements this learning by providing end-to-end placement assistance, facilitating connections with over 500 recruiting companies including MNCs and startups. This ecosystem-centric approach ensures that learners are not just trained but are also empowered to enter the job market with confidence and competence.

A key differentiator in CETPA's model is its emphasis on academic-industry integration. The company has partnered with numerous engineering and management institutions to deliver on-campus training, internships, and faculty development programs. This enables a seamless transition for students from classroom learning to corporate environments. CETPA also supports corporate training initiatives and government skill development programs, reflecting its commitment to inclusive and scalable talent development. By leveraging experienced trainers, domain-specific mentors, and real-time analytics, the platform delivers customized learning paths tailored to individual and institutional goals.

CETPA's inclusive approach extends beyond training. The organization also functions as a software and embedded system development partner, delivering enterprise-grade solutions in areas such as web and mobile applications, cybersecurity, cloud computing, automation systems, and IoT. This dual role of educator and solution provider reinforces CETPA's value proposition as a technology enabler across both learning and implementation domains.

As technology continues to redefine professional competencies and organizational needs, CETPA Infotech stands out as a critical enabler in the Indian tech education ecosystem. Its commitment to bridging the skill gap through quality training, innovation, and industry collaboration makes it a pivotal contributor to workforce development, employability enhancement, and the broader knowledge economy.

CHAPTER NO. 1

INTRODUCTION

1. INTRODUCTION

1.1 PROJECT SUMMARY

The project titled Bank Management System is a web-based application developed to automate and simplify essential banking operations for both administrators and users. Designed and implemented using Java programming language with MySQL as the backend database, the system provides a secure and user-friendly interface to manage various aspects of banking, such as account creation, transaction processing, fund transfers, and report generation. The project adheres to the MVC (Model-View-Controller) architecture to ensure modularity, maintainability, and scalability.

The system comprises two main user roles: Admin and User. The Admin module allows authorized personnel to perform operations like adding, updating, and deleting accounts, viewing user details, managing active or inactive accounts, and generating transaction statements. The User module allows customers to securely log in using a PIN system and perform activities such as checking account balances, depositing and withdrawing funds, transferring money domestically or internationally, and accessing detailed transaction histories.

This application addresses the limitations of manual banking systems, such as long processing times, limited accessibility, and risk of data loss or human error. By offering 24/7 availability through a web portal, it brings convenience to users and improves operational efficiency for banks.

Furthermore, the Bank Management System project highlights the broader vision of transforming traditional banking into a fully digital and paperless model. It supports future scalability by enabling the development of a mobile banking application and the inclusion of advanced analytics, CRM integration, and customer feedback modules. The training and development support received from CETPA Infotech Pvt. Ltd. helped in shaping this industry-relevant solution. This project serves as a practical demonstration of applying theoretical computer science knowledge to real-world financial technology solutions and prepares the developer for future challenges in enterprise-level software development.

1.2 PROJECT OBJECTIVE

The Bank Management System project aims to create a secure, efficient, and user-friendly platform that automates day-to-day banking operations. With the growing demand for digital banking, this system is designed to replace manual and time-consuming processes with an online solution that benefits both bank administrators and customers. Developed using Java and MySQL, and based on the MVC architecture, this project reflects the modern needs of a responsive and scalable banking environment. The objectives of the project are outlined below:

1. **To develop a web-based banking system** that facilitates essential functionalities such as account creation, balance enquiry, fund transfers, deposit, and withdrawal in a seamless manner.
2. **To offer round-the-clock banking services to users**, allowing them to manage their accounts securely from remote locations using a browser-based interface.
3. **To build a modular and maintainable system** using MVC (Model-View-Controller) architecture that separates concerns and supports future scalability and upgrades.
4. **To implement role-based access control**, allowing administrators to manage user data and transactions, and customers to access their individual accounts securely.
5. **To ensure data security and privacy** through login authentication, session handling, and secure transaction processing mechanisms.
6. **To maintain accurate transaction records** and enable real-time updates to account balances, ensuring transparency and trust in financial operations.
7. **To reduce operational load on banking staff** by automating repetitive tasks and minimizing the dependency on manual record keeping.

1.3 TOOLS/ PLATFORM HARDWARE & SOFTWARE REQUIREMENTS

The **Bank Management System** is developed using a modern full-stack technology framework, ensuring scalability, security, and maintainability. The following tools and platforms were used:

Hardware Requirements

Component	Minimum Specification
Processor	Intel Core i5 / AMD Ryzen 5 or higher
RAM	8 GB (16 GB recommended for smooth build/run)
Storage	256 GB SSD or 500 GB HDD
Display	13" or larger monitor (Full HD preferred)
Internet Connection	Required for database access, version control, and deployment

Software Requirements

Software	Version / Description
Operating System	Windows 10 or higher
Java Development Kit (JDK)	Version 11 or higher
IDE	Eclipse/IntelliJ IDEA/Spring Tool Suite
MYSQL	MySQL Server 8.0 or higher
SQL Server Management Studio (SSMS)	Latest version
Git	Latest version
Web Browser	Google Chrome / Microsoft Edge (latest)

1.4 PROJECT CATEGORY

The Bank Management System project falls under the category of **Web-Based Enterprise Applications** with a strong orientation towards **Financial and Transactional Software Systems**. As an enterprise-level solution, it is designed to address the real-world needs of digital banking institutions by automating core banking operations and delivering seamless service to both administrators and customers. The project is a combination of software engineering, database management, web development, and secure financial transaction processing, making it a multi-domain application. It involves both front-end and back-end development and is implemented using modern programming practices such as MVC (Model-View-Controller) architecture, object-oriented design, and relational database systems. This system simulates the operations typically carried out in a banking environment such as creating customer accounts, processing deposits and withdrawals, enabling fund transfers, managing account information, and generating transaction reports. Being an interactive, role-based system, it serves as a practical demonstration of building scalable, secure, and real-time applications. The project showcases the use of Java and Spring Framework on the backend, MySQL as the database, and Thymeleaf for dynamic content rendering. It emphasizes user authentication, data integrity, and modular development, qualifying it as a secure, web-based enterprise project suitable for deployment in real-life scenarios such as cooperative banks, digital wallets, or internal banking systems. Thus, the project not only serves as an academic software solution but also aligns with the standards and expectations of real-world fintech applications.

CHAPTER NO. 2

SYSTEM STUDY

2. SYSTEM STUDY

The process of system study plays a vital role in the development of any software application, particularly in complex and sensitive domains like banking. System study involves understanding, analyzing, and documenting the existing operational procedures and user requirements to develop an effective, efficient, and reliable system. In the case of the Bank Management System, the system study is essential for identifying the challenges faced by traditional banking operations and determining how technology can be employed to enhance banking services for both administrators and end users.

Banking is one of the most crucial sectors of the economy, dealing with sensitive information and real-time financial transactions. Traditionally, banks have relied on manual processes involving paperwork, physical records, and customer visits for activities such as account creation, deposits, withdrawals, balance enquiries, and fund transfers. These manual systems are often time-consuming, error-prone, and inefficient, leading to delays, customer dissatisfaction, and increased operational overhead. Furthermore, with the rapid rise of digital banking and increased customer expectations, banks must transition to robust online systems that can perform financial operations securely and efficiently.

System study in this context begins by analyzing the functional requirements of both the bank's administration and its customers. This includes studying how user data is collected, stored, and updated, how transactions are initiated and completed, and how reports are generated. It also considers the administrative tasks like managing user accounts, monitoring transaction histories, and handling inactive or fraudulent accounts. The goal is to ensure the new system provides real-time, accurate, and secure services while improving the overall user experience and reducing operational costs.

Additionally, the system study explores technical requirements such as the appropriate development platform, database management system, user interface design, system architecture, and security protocols. It also addresses performance needs such as system speed, concurrency, and scalability, as banking applications often deal with high volumes of data and users simultaneously. The study involves feedback from end users and domain experts to ensure the final product meets real-world expectations.

and regulatory standards.

By conducting a thorough system study, the *Bank Management System* project lays a strong foundation for building a software solution that mirrors the real-time processes of a financial institution while eliminating the shortcomings of manual systems. This phase ensures that the system is not only technically sound but also aligned with the organizational goals of speed, accuracy, security, and customer satisfaction.

2.1 PRELIMINARY INVESTIGATION

The preliminary investigation is the initial phase of the software development life cycle (SDLC) and serves as the foundation for understanding the problem domain and evaluating the feasibility of developing a proposed system. In the context of the Bank Management System, this stage involves identifying the current issues in traditional banking workflows and exploring how a digital solution can address these limitations efficiently and securely.

The existing manual banking systems often rely heavily on physical records, face-to-face interactions, and time-consuming paperwork. Customers need to visit the bank for even minor tasks such as balance inquiries, fund transfers, or account updates. This results in unnecessary delays, overcrowding in bank branches, a higher chance of human error, and inefficient service delivery. Moreover, the lack of centralized digital tracking makes it difficult to maintain transaction histories, user authentication, and real-time data processing. These shortcomings highlight the pressing need for a technology-driven, user-friendly, and secure banking application.

During the preliminary investigation of the proposed Bank Management System, several key objectives were identified. These include simplifying daily banking operations, reducing dependency on physical infrastructure, enabling secure user login and transaction handling, and providing real-time access to account information. The investigation also focused on ensuring data consistency and accuracy through a centralized database (MySQL), implementing a scalable system architecture (MVC), and enhancing user accessibility via a web interface.

Additionally, this phase included stakeholder discussions, requirement gathering, and reviewing existing systems and user pain points. It became evident that both administrators and customers would benefit significantly from a system that supports online account management, secure transactions, and detailed reporting features.

Based on this initial investigation, the project was found to be **technically feasible**, **economically viable**, and **operationally necessary**. It lays the groundwork for a complete analysis and design process that will lead to the successful implementation of a comprehensive and secure banking system capable of meeting modern-day digital requirements.

2.2. SYSTEM DEVELOPMENT LIFE CYCLE

The System Development Life Cycle (SDLC) is a structured and methodical process followed to build high-quality software systems. It ensures that each phase of the software development process is thoroughly planned, executed, and validated. For the Bank Management System, the following SDLC phases were followed to achieve a secure, functional, and user-friendly banking solution.

1. Requirement Gathering and Analysis

During this initial phase, both functional and non-functional requirements were gathered through observation, stakeholder feedback, and study of traditional banking workflows. Key requirements such as account registration, deposit/withdrawal operations, transaction history, user authentication, and administrative controls were identified. Separate needs were collected for admin and user roles to ensure a clear understanding of different functionalities. The limitations of manual banking were studied to guide the direction of automation and system improvement.

2. System Design

Based on the collected requirements, the system's technical and architectural design was prepared. The software was designed using the MVC (Model-View-Controller) architecture, enabling modular development and easy maintenance. The design phase included creating the database schema using MySQL, defining the relationships between entities like User, Account, and Transaction, and designing wireframes and navigation flows for the web interface. A detailed Entity Relationship Diagram (ERD) and Data Flow Diagram (DFD) were also developed to ensure clarity in system structure.

3. Implementation

The implementation phase involved developing the application using **Java (Spring Framework)** for the backend and **MySQL** for the database. The frontend was created using HTML, CSS, Thymeleaf, and Bootstrap to ensure a responsive and interactive user experience. Features such as secure login, deposit/withdraw forms, fund transfers, balance enquiries, and transaction reporting were implemented with role-based access control. Controllers and services were coded to handle business logic and connect to the database efficiently.

4. Testing

Various types of testing were conducted to ensure the system worked as intended. Unit testing was performed on individual service and controller methods, while integration testing checked the end-to-end functioning of transaction modules. User acceptance testing (UAT) was also carried out to confirm that both admins and users could navigate the system easily and perform actions without errors. The application was validated for data accuracy, security, and proper session handling under different usage scenarios.

2.3 FEASIBILITY STUDY

The feasibility study is a crucial part of the software development life cycle that determines whether the proposed system is viable, practical, and beneficial for implementation.

From a **technical feasibility** perspective, the project was found to be highly practical due to the availability of reliable and mature technologies such as Java, Spring Framework, MySQL, HTML, and CSS. These tools are well-supported, platform-independent, and suitable for building secure and scalable enterprise applications. Additionally, the use of the MVC architecture ensured better separation of concerns and maintainability, while the chosen hardware specifications were affordable and compatible with the system's requirements.

In terms of **operational feasibility**, the system was designed with end-users in mind—both administrators and customers—ensuring ease of use through a simple and responsive user interface. The application allows customers to perform day-to-day banking tasks such as balance enquiries, fund transfers, and transaction history tracking without visiting the bank, while administrators can efficiently manage user accounts, transactions, and reports. These capabilities significantly reduce manual effort, improve customer satisfaction, and enhance operational productivity within the banking system.

The **economic feasibility** was also carefully evaluated. The development and deployment of this system require minimal cost as it leverages open-source technologies and does not depend on expensive licensed software. The reduction in manual processing, paperwork, and human errors also translates to cost savings for the organization in the long term. Furthermore, the system provides high return on investment by offering continuous online services, reducing physical infrastructure needs, and enabling faster customer servicing.

Overall, the feasibility study concluded that the *Bank Management System* is a technically sound, operationally useful, and economically viable project. It not only addresses the limitations of the existing manual banking process but also lays a strong foundation for further enhancements such as mobile banking integration, data analytics, and advanced customer relationship management systems.

2.3.1 Technical feasibility

The Technical feasibility of the Bank Management System evaluates whether the current technology, tools, and infrastructure are capable of supporting the successful development and implementation of the system.

Technology Stack: The system is developed using Java (with Spring Framework) for backend logic, MySQL as the relational database, and HTML, CSS, Thymeleaf, and Bootstrap for the frontend. All these tools are open-source, reliable, and widely adopted in enterprise-level web applications.

Architecture: The project follows the MVC (Model-View-Controller) architecture, allowing clear separation of concerns, modular development, easier debugging, and future scalability.

Development Tools: Tools like Eclipse or IntelliJ IDEA, Apache Tomcat Server, and MySQL Workbench are used, which are compatible with the chosen technology stack and provide a smooth development environment.

Hardware Compatibility: The application can run efficiently on standard hardware configurations such as Intel i5 processor, 8 GB RAM, and 256 GB storage, making it feasible for deployment on most systems without additional cost.

Security & Performance: Secure session handling, role-based access control, and ACID-compliant database transactions ensure that the system can handle financial operations reliably and securely with good performance.

2.3.2 Operational feasibility

Operational feasibility assesses how effectively the proposed system will function in a real-world environment and whether it can be successfully integrated into the daily operations of the organization. For the Bank Management System, the main objective is to simplify banking processes, enhance user experience, and reduce dependency on manual procedures. It is essential that the system is easy to use, reliable in handling financial transactions, and aligns well with the operational workflow of both bank staff and customers. Based on the system's design and features, the following points demonstrate its strong operational feasibility:

User-Friendly Interface: Designed with a clean and responsive layout using HTML, CSS, Bootstrap, and Thymeleaf, the system ensures ease of use for both administrators and customers, even with minimal technical knowledge.

Role-Based Access Control: The system separates user roles clearly, enabling customers to manage personal banking activities while allowing administrators to oversee and manage account-level operations and transactions efficiently.

Time and Resource Efficiency: By providing online access to banking services like fund transfers, balance inquiries, and transaction history, the system reduces queue time and reliance on branch visits, improving overall service delivery.

Error Reduction and Accuracy: Automating the processing of data and transactions significantly minimizes the risk of human errors, leading to more accurate records and smoother banking operations.

Quick Adoption and Low Maintenance Overhead: The system is easy to deploy, requires minimal staff training, and supports straightforward maintenance, making it suitable for long-term use in any bank setting.

2.3.3 Economic feasibility

Economic feasibility evaluates the cost-effectiveness of the proposed system by analyzing whether the benefits of the system justify the financial investment required for its development, implementation, and maintenance. For the *Bank Management System*, the goal is to deliver a high-performing, secure, and user-friendly solution using affordable technologies and resources. Since the project uses open-source tools and requires no expensive proprietary licenses, the overall cost of development is significantly minimized. In addition, automation leads to long-term savings by reducing manual effort, paper usage, and operational overhead. The following points summarize the system's strong economic viability:

- **Low Development Cost:** The system is built using open-source technologies such as Java, Spring Framework, MySQL, and Apache Tomcat, eliminating the need for costly licenses or commercial development tools.
- **Minimal Hardware Requirements:** The application can run on standard computers with moderate specifications (Intel i5, 8 GB RAM), avoiding the need for specialized or high-end hardware.
- **Reduced Operational Costs:** By digitizing common banking functions like deposits, withdrawals, and account management, the system reduces paperwork, administrative workload, and the need for physical storage and printing.
- **High Return on Investment (ROI):** Once deployed, the system provides continuous online service, and enhances customer satisfaction—leading to long-term cost savings and value generation for the bank.
- **Scalability without Major Expense:** As the system is designed with scalability in mind, it can be easily upgraded or integrated with future features like mobile apps or analytics modules without incurring significant additional costs.

CHAPTER NO. 3
SYSTEM ANALYSIS

3. SYSTEM ANALYSIS

System Analysis is a vital phase in the development of the Bank Management System, as it focuses on translating banking requirements into clearly defined technical specifications. This phase involves a thorough examination of the current manual processes within a bank, identifying gaps, inefficiencies, and pain points, and proposing a structured solution that automates key operations. The goal is to ensure that the final system effectively supports customer and administrative needs while being reliable, secure, and scalable. The following steps outline the activities carried out during the system analysis phase:

1. Requirements Gathering

Detailed information was collected from potential end-users, including bank staff (administrators) and customers. Functional requirements such as user registration, login with PIN, deposits, withdrawals, fund transfers, transaction history viewing, and admin-level account control were documented. Non-functional requirements including system security, real-time processing, database integrity, and platform compatibility were also identified.

2. Feasibility Study

A feasibility study was conducted based on the collected requirements to ensure that the system was viable in all respects. Technical feasibility confirmed the availability of open-source tools like Java and MySQL. Operational feasibility validated ease of use for both customers and staff. Economic feasibility confirmed that the project could be developed at low cost with long-term benefits. Legal and data protection compliance were also considered.

3. System Design

After confirming feasibility, the system architecture was designed using the MVC (Model-View-Controller) approach. The backend was implemented in Java using Spring Framework, the frontend was structured with HTML/CSS and Thymeleaf, and MySQL was selected for database management. Wireframes were created for UI planning, and system modules were outlined for both user and admin roles.

4. Data Flow and Process Modeling

Key diagrams such as the Data Flow Diagram (DFD) and Entity Relationship Diagram (ERD) were created to visualize system functionality. The DFD mapped user interactions such as login, deposit, withdrawal, and transfer, while the ERD described relationships between entities like User, Account, and Transaction. These models ensured a clear understanding of how data moves across the system.

5. Identify System Requirements

Hardware and software specifications were identified to support the system's operations. This included Intel i5+ processors, 8 GB RAM, and standard storage along with software such as Java JDK 11+, Apache Tomcat, MySQL Server, and IntelliJ/Eclipse IDE. System performance requirements were defined, including real-time transaction updates, multi-user support, and session handling.

6. Problem Identification and Solutions

The existing banking process involved manual data entry, in-person transactions, and delays in report generation. These problems were addressed by automating processes, enabling online access to banking services, integrating account-level validations, and implementing secure authentication. The system aimed to reduce human error, save time, and enhance service reliability.

7. System Validation

The system design and features were validated with user feedback to ensure that the application met both functional and operational needs. Test cases were designed to confirm the correctness of transaction handling, user login, and role-based access. Feedback loops during the design phase allowed for refinement before implementation.

8. Risk Assessment

Risks such as data breaches, technical limitations, and poor user adoption were identified. To mitigate these, role-based access control, secure session management, regular backups, and user training were planned. Scalability was also considered to allow future upgrades without system failure.

9. Documentation

All requirements, designs, diagrams, and specifications were thoroughly documented to guide development and testing. This documentation also serves as a reference for future maintenance, scalability improvements, and system audits.

10. Project Timeline and Resource Allocation

A project plan was prepared outlining phases like system design, development, testing, and deployment. Human resources, such as backend developers, frontend designers, and QA testers, were allocated, along with required hardware and software resources. Budget considerations included development time, training, and hosting costs.

3.1 EXISTING SYSTEM

The Existing Banking System in many organizations still relies heavily on manual and semi-automated processes, which create significant limitations in terms of speed, accuracy, and user convenience. In the traditional system, customers are required to physically visit the bank to perform basic operations such as opening an account, depositing or withdrawing money, requesting account statements, or transferring funds. These operations are often recorded on paper or stored in outdated local software, which leads to risks of human error, data inconsistency, and security breaches. Additionally, the dependency on manual procedures results in longer processing times, lack of real-time updates, and inefficient customer service. Bank staff must spend considerable time handling paperwork, managing ledgers, and maintaining transaction logs, which reduces overall productivity. The absence of centralized data management makes it difficult to track transaction history, monitor suspicious activity, or generate reports instantly. Furthermore, the lack of remote access prevents customers from managing their accounts outside banking hours or from distant locations. In today's fast-paced and digitally driven world, the existing system fails to meet modern banking demands, making it essential to adopt a secure, automated, and user-friendly online banking solution that enhances service delivery and operational efficiency.

3.2 PROPOSED SYSTEM

The Proposed Bank Management System is a web-based software application designed to overcome the limitations of the existing manual banking system by offering fast, secure, and user-friendly online banking services. The system aims to automate essential banking operations such as account creation, deposits, withdrawals, fund transfers, and transaction history tracking through a centralized digital platform. Developed using Java (Spring Framework) and MySQL, the system ensures real-time transaction processing, secure data handling, and seamless communication between users and administrators. The platform adopts the Model-View-Controller (MVC) architecture, which enhances modularity, simplifies development, and allows for easy future upgrades.

The application features two main modules: the **User Module** and the **Admin Module**. The User Module enables customers to register, log in securely with a PIN system, view their balance, deposit or withdraw funds, transfer money to other accounts, and track transaction history with just a few clicks. On the other hand, the Admin Module provides functionalities for managing customer records, activating or deactivating accounts, viewing all transactions, generating reports, and overseeing the overall banking operations. This role-based access ensures that sensitive tasks are handled with appropriate privileges, improving data integrity and system security.

With a responsive and intuitive user interface built using HTML, CSS, Thymeleaf, and Bootstrap, the system ensures accessibility across various devices and platforms. It minimizes human error, eliminates paperwork, and reduces customer dependency on physical branch visits. In addition, the proposed system improves operational efficiency, reduces service time, and enables round-the-clock access to banking services. Enhanced security measures such as session management, validation checks, and controlled access ensure the safety of sensitive financial data. The proposed system is also scalable, allowing for future integration with mobile banking apps, automated notifications, and advanced analytics tools. In summary, the proposed *Bank Management System* provides a modern, comprehensive, and efficient solution for digitalizing core banking activities in line with current technological standards and user expectations.

CHAPTER NO. 4

SYSTEM DESIGN

4. SYSTEM DESIGN

System Design for the Bank Management System defines the overall architecture, components, data flow, and interfaces that ensure the development of a scalable, secure, and efficient digital banking platform. It is divided into two main parts: **High-Level Design (HLD)** and **Low-Level Design (LLD)** to provide a clear understanding of both the structural layout and internal workings of the system.

1. High-Level Design (HLD):

This defines the architecture and major components of the system:

- **User Interface Layer:** A web-based front end for Admins and Customers, built using HTML, CSS, Bootstrap, and Thymeleaf. It allows users to interact with the system via features like login, balance check, transactions, and account management.
- **Business Logic Layer:** Developed using Java and Spring Framework, this layer handles core operations such as user authentication, deposit, withdrawal, fund transfer, and transaction validation.
- **Data Access Layer:** Interfaces with the MySQL database to store and retrieve user data, account details, and transaction history using repositories and service classes.
- **Authentication Module:** Implements secure login and PIN verification for customers and credentials-based access for admins, enforcing role-based access control to separate functionalities and restrict unauthorized actions.
- **Reporting Module:** Enables users to view past transactions and allows admins to generate reports for account activity and system logs.
- **Error Handling & Session Management:** Tracks invalid actions, manages user sessions securely, and provides real-time error messages for invalid inputs or system failures.

2. Low-Level Design (LLD):

This section details the individual module functionalities and internal workflows:

- **User Registration and Login:** Customers can register and log in using a secure PIN; credentials are verified against the database with encrypted session handling.
- **Deposit and Withdrawal Modules:** Allows users to deposit and withdraw money. Validations ensure amounts are within allowed limits and balances are updated instantly.
- **Fund Transfer:** Supports internal account transfers by validating receiver details and checking for sufficient balance before processing the transaction.
- **Transaction History:** Users can view a detailed list of their past transactions with filters for date and type. Admins can monitor all transactions system-wide.
- **Admin Dashboard:** Admins can add or remove customer accounts, activate/deactivate users, view logs, and manage data integrity.
- **Balance Enquiry:** A real-time feature that retrieves the current balance from the user's account and displays it instantly.

Design Principles:

- **Modular & Scalable:** The system is divided into distinct layers and services, making it easy to maintain, debug, and extend with future features such as loan modules or mobile apps.

- **Secure:** Includes session validation, encrypted login credentials, PIN-based user verification, and access control to ensure data protection.
- **User-Friendly UI:** Clean, minimal interface using Bootstrap and Thymeleaf, ensuring a responsive and intuitive experience across devices.

4.1 ENTITY RELATIONSHIP DIAGRAM (ERD)

1. Admin

- Attributes: AdminID (PK), Name, Email, Password
- Relationships:
 - Manages → User
 - Monitors → Transaction
 - Controls → Account

2. User

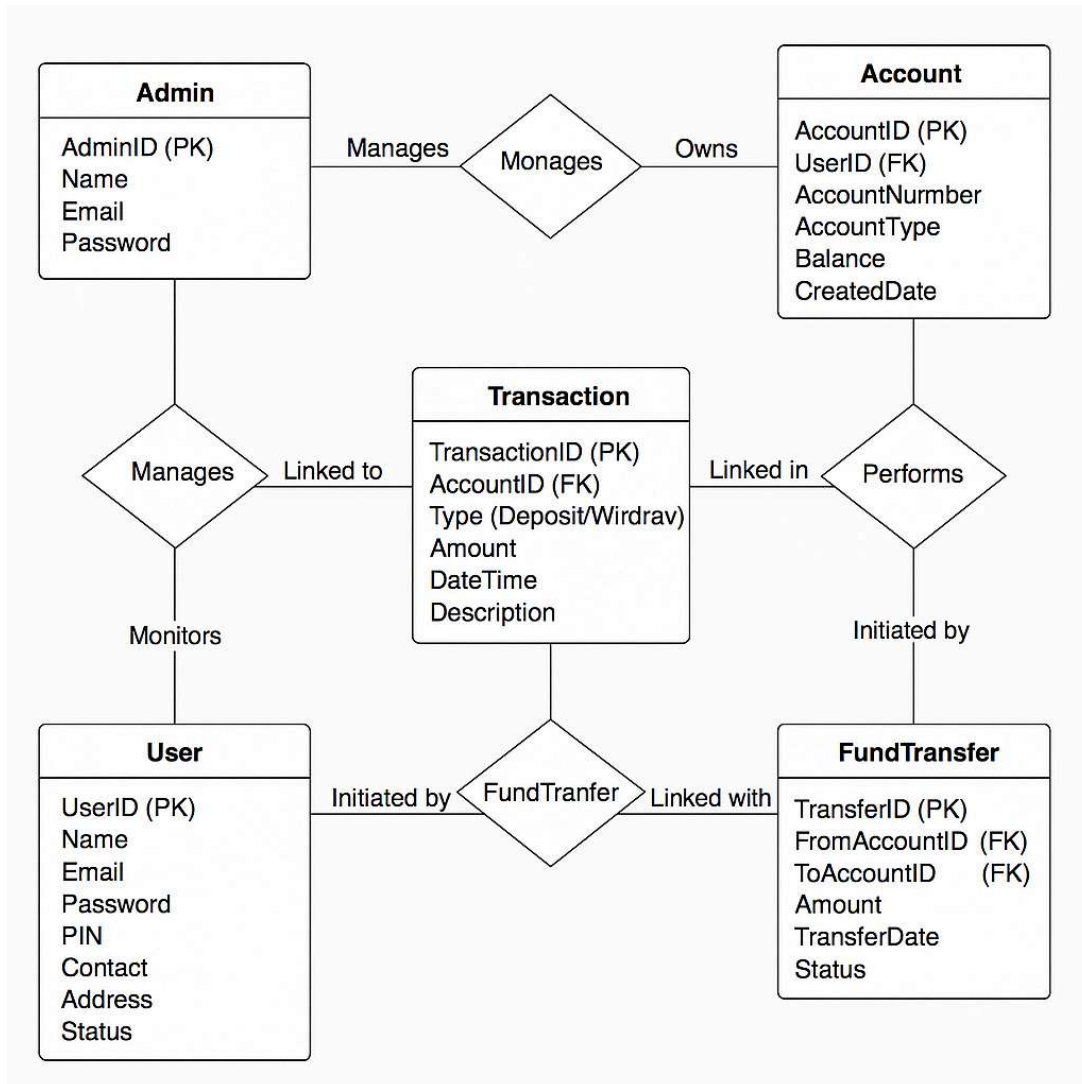
- Attributes: UserID (PK), Name, Email, Password, PIN, Contact, Address, Status
- Relationships:
 - Owns → Account
 - Performs → Transaction
 - Requests → FundTransfer

3. Account

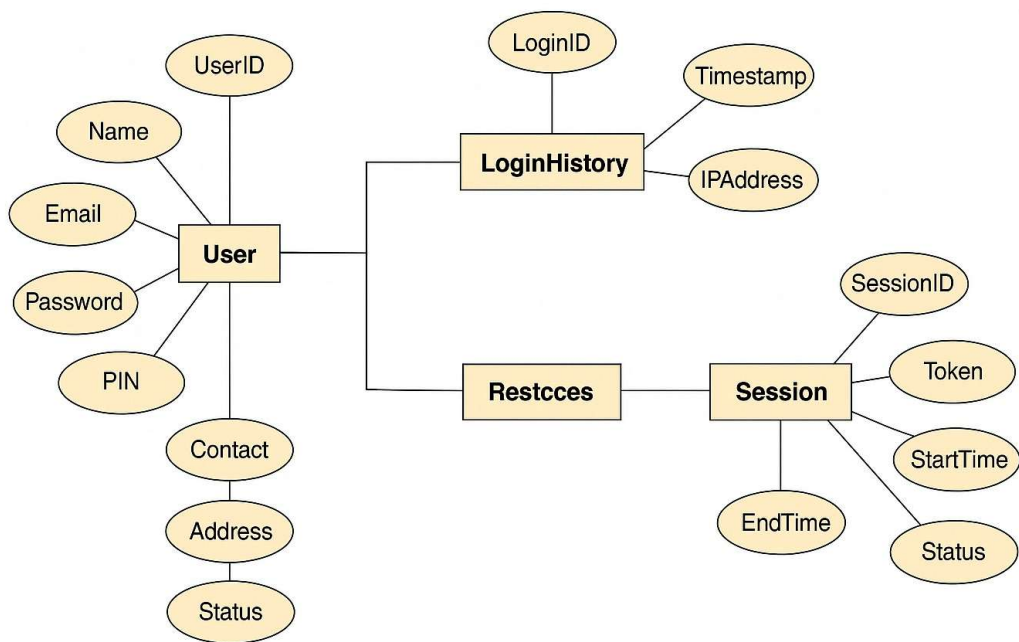
- Attributes: AccountID (PK), UserID (FK), AccountNumber, AccountType, Balance, CreatedDate
- Relationships:
 - Linked to → User
 - Involved in → Transaction
 - Source/Destination in → FundTransfer

4. Transaction

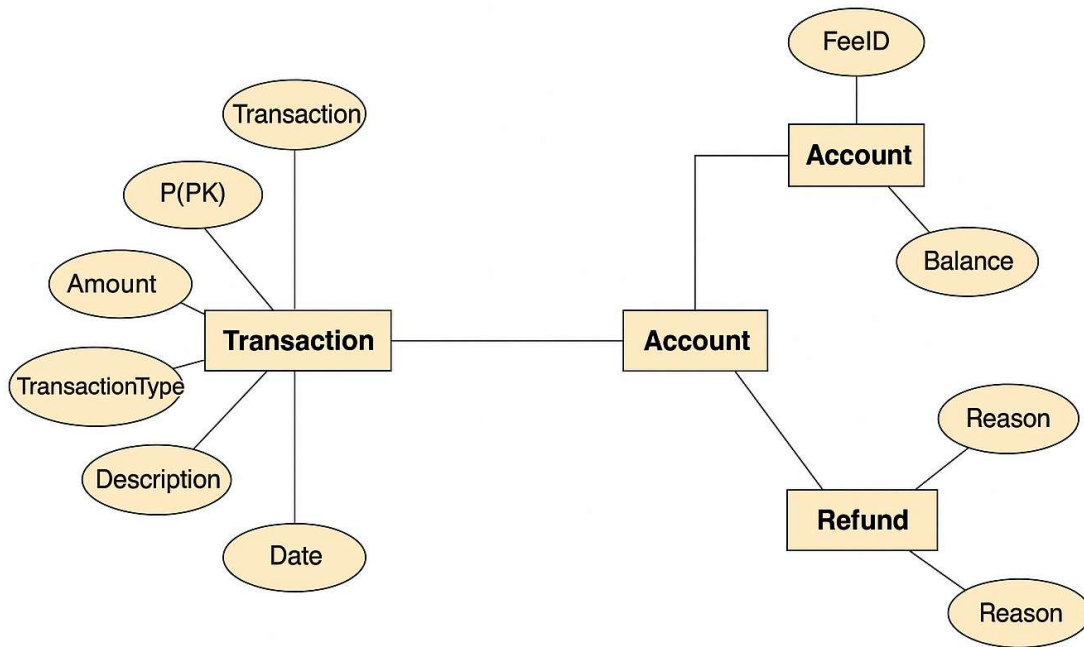
- Attributes: TransactionID(PK), AccountID(FK), Type (Deposit/Withdraw), Amount, DateTime, Description
- Relationships:
 - Belongs to → Account
 - Recorded by → Admin



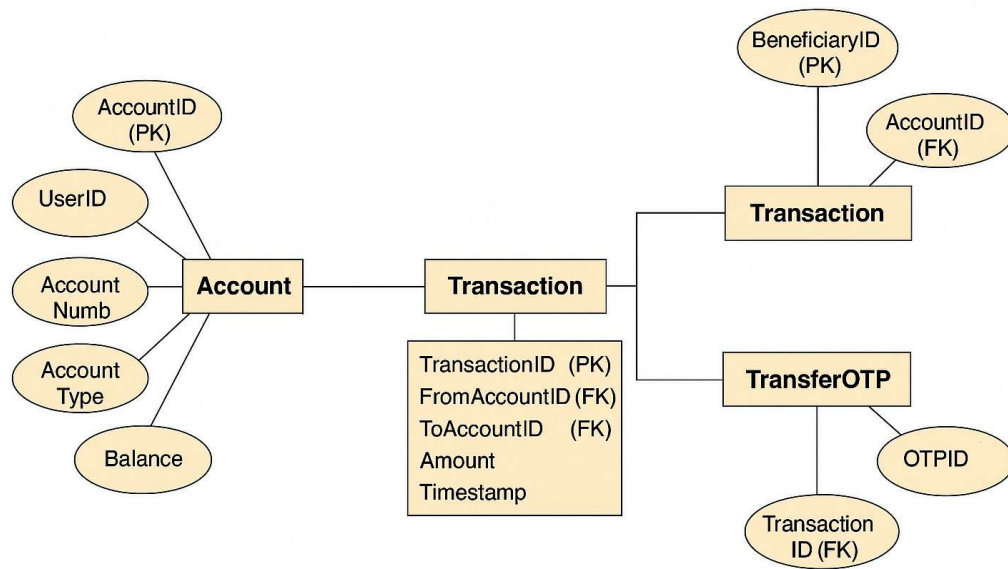
ER diagram for Bank Management System



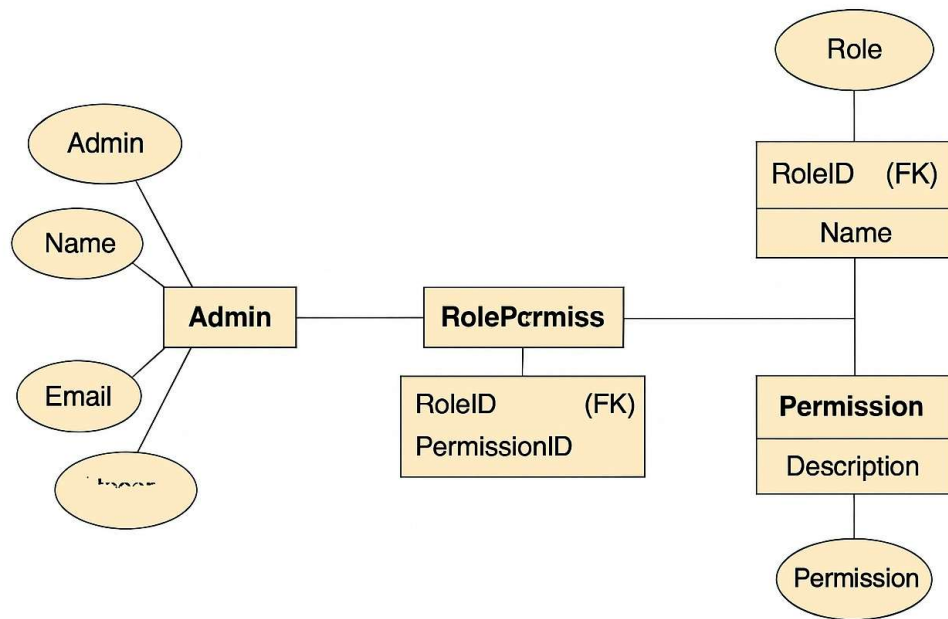
User Management Module



Transaction Management Module



Fund Transfer Module



Admin Management Module

CHAPTER NO. 5
SYSTEM DEVELOPMENT

5. SYSTEM DEVELOPMENT

5.1 CODING

Balance And Statement Controller Code :

```
package com.jantabank.controllers; import java.util.List;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.stereotype.Controller;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.SessionAttribute;
import org.springframework.web.servlet.ModelAndView;
import com.jantabank.entities.TransactionInfo;
import com.jantabank.repositories.TransactionRepository;
import com.jantabank.services.AccountService;
@Controller
public class BalanceAndStatementController
{
    @Autowired private AccountService accountService;
    @Autowired private TransactionRepository transactionRepository;
    @GetMapping("services/view-balance")
    public ModelAndView getAccountBalance(@SessionAttribute int accountno)
    {
        int amount=accountService.getAmount(accountno);
        ModelAndView modelAndView=new
ModelAndView("services/balance/show-balance");
        modelAndView.addObject("amount",amount);
        return modelAndView;
    }
    @GetMapping("services/view-statements")
    public ModelAndView getStatementOfCurrentUser(@SessionAttribute int
accountno)
    {
        List<TransactionInfo>
statements=transactionRepository.getTransactionListOfCurrentUser(accountno);
```

```

        ModelAndView modelAndView=new
ModelAndView("services/balance/show-statements");
        modelAndView.addObject("statementList",statements);
        return modelAndView;
    }
}

```

Deposit Money Controller :

```

package com.jantabank.controllers;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.stereotype.Controller;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.ModelAttribute;
import org.springframework.web.bind.annotation.SessionAttribute;
import org.springframework.web.servlet.ModelAndView;
import org.springframework.web.servlet.mvc.support.RedirectAttributes;
import com.jantabank.entities.TransactionInfo;
import com.jantabank.services.AccountService;

```

@Controller

public class DepositMoneyController

{

 @Autowired private AccountService accountService;

 @GetMapping("services/deposit-form")

 public String getDepositForm()

 {

 return "services/deposit/deposit-money-form";

 }

 @GetMapping("services/deposit.do")

 public String depositMoney(int amount,@SessionAttribute("accountno") int
accountno,RedirectAttributes
redirectAttributes)

```

{
    TransactionInfo transaction=accountService.creditAmount(amount,accountno);
    redirectAttributes.addFlashAttribute("transactionInfo", transaction);
    return "redirect:deposit-success";
}
@GetMapping("services/deposit-success")
public ModelAndView depositMoneySuccess(@ModelAttribute("transactionInfo")
TransactionInfo
transaction)
{
    ModelAndView modelAndView=new
ModelAndView("services/deposit/deposit-money-success");
    modelAndView.addObject("transaction", transaction);
    return modelAndView;
}
}

```

Home Controller :

```

package com.jantabank.controllers;
import org.springframework.stereotype.Controller;
import org.springframework.web.bind.annotation.GetMapping;
import jakarta.servlet.http.HttpSession;
@Controller
public class HomeController
{
    @GetMapping
    public String getHomeView(HttpSession session)
    {
        System.out.println("Home controller");
        if(session.getAttribute("username")==null)
            return "user/login/login-form";
        return "home/home-page";
    }
}

```

```
}
```

Transfer Controller :

```
package com.jantabank.controllers;
import jakarta.servlet.http.HttpSession;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.stereotype.Controller;
import org.springframework.ui.Model;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.ModelAttribute;
import org.springframework.web.servlet.ModelAndView;
import org.springframework.web.servlet.mvc.support.RedirectAttributes;
import com.jantabank.entities.TransactionInfo;
import com.jantabank.services.AccountService;
```

```
@Controller
```

```
public class TransferController
```

```
{
```

```
    @Autowired private AccountService accountService;
```

```
    @GetMapping("services/transfer-form")
```

```
    public String getTransferFormView()
```

```
    {
```

```
        return "services/transfer/transfer-money-form";
```

```
    }
```

```
    @GetMapping("services/verify-account.do")
```

```
    public ModelAndView verifyAccountNo(int accountno, HttpSession session)
```

```
    {
```

```
        ModelAndView modelAndView=new
```

```
ModelAndView("services/transfer/transfer-money-form");
```

```
        session.setAttribute("raccountno",accountno);
```

```
        if((Integer)session.getAttribute("accountno")==accountno)
```

```

    {
        modelAndView.addObject("msg","This is your own account number");
        return modelAndView;
    }
    String name=accountService.getAccountHolder(accountno);
    if(name==null)
    {
        modelAndView.addObject("msg","Entered account number does not exist");
        return modelAndView;
    }
    modelAndView.setViewName("services/transfer/transfer-money-form2");
    session.setAttribute("rname",name);
    return modelAndView;
}
@GetMapping("services/transfer.do")
public String performTransferMoneyOperation(int amount,HttpSession
httpSession,RedirectAttributes
redirectAttributes,Model model)
{
    int accountno=(Integer)httpSession.getAttribute("accountno");
    int accountBalance=accountService.getAmount(accountno);
    if(amount>accountBalance)
    {
        model.addAttribute("msg","You do not have sufficient amount into your
account");
        model.addAttribute("amount",amount);
        return "services/transfer/transfer-money-form2";
    }
    TransactionInfo
transaction=accountService.transferMoney(amount,accountno,(Integer)httpSession.ge
tAttribute("raccountno"));
    redirectAttributes.addFlashAttribute("transaction", transaction);
    return "redirect:transfer-success";
}

```



```

    @GetMapping("services/transfer-success")
    public ModelAndView transferMoneySuccess(@ModelAttribute("transaction")
TransactionInfo transaction)
    {
        ModelAndView modelAndView=new
ModelAndView("services/transfer/transfer-money-success");
        modelAndView.addObject("transaction",transaction);
        return modelAndView;
    }
}

```

User Controller :

```

package com.jantabank.controllers;
import jakarta.servlet.http.HttpSession;
import java.time.LocalDate;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.stereotype.Controller;
import org.springframework.ui.Model;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.PostMapping;
import org.springframework.web.servlet.ModelAndView;
import com.jantabank.entities.Account;
import com.jantabank.entities.User;
import com.jantabank.services.AccountService;
import com.jantabank.services.UserService;

```

```

@Controller
public class UserController
{
    @Autowired private UserService userService;
    @Autowired private AccountService accountService;

    /*@GetMapping("user/login")
    public String getLoginForm()

```

```

{
    return "user/login/login-form";
}*/
@GetMapping("user/registration")
public String getRegistrationForm(Model model)
{
    model.addAttribute("cdate",LocalDate.now());
    return "user/registration/registration-form";
}
@PostMapping("user/register-user")
public ModelAndView registerUserAndCreateAccount(User user)
{
    int accountno=userService.saveUserAndCreateAccount(user);
                                ModelAndView modelAndView=new
ModelAndView("user/registration/registration-success");
                                modelAndView.addObject("username",user.getFirstname()+"
"+user.getLastname());
    modelAndView.addObject("accountno",accountno);
    return modelAndView;
}
@PostMapping("user/authenticate-user")
    public String authenticateUser(String userid,String password,Model
model,HttpSession session)
{
    User user=userService.getUser(userid);
    if(user==null)

    {
        model.addAttribute("msg","Entered userid does not exist");
        model.addAttribute("uid",userid);
        return "user/login/login-form";
    }
    String dbpassword=user.getPassword();
    if(!password.equals(dbpassword))

```

```

    {
        model.addAttribute("msg","Entered password is wrong");
        model.addAttribute("uid",userid);
        return "user/login/login-form";
    }
    session.setAttribute("username",user.getFirstname()+" "+user.getLastname());
    Account account=accountService.getAccountById(userid);
    session.setAttribute("accountno",account.getAccountno());
    return "redirect:/";
}
@GetMapping("user/logout")
public String userLogout(HttpSession session,Model model)
{
    model.addAttribute("username",session.getAttribute("username"));
    session.invalidate();
    return "user/login/logout-success";
}
}

```

Withdraw Money Controller :

```

package com.jantabank.controllers;

import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.stereotype.Controller;
import org.springframework.ui.Model;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.ModelAttribute;
import org.springframework.web.bind.annotation.SessionAttribute;
import org.springframework.web.servlet.ModelAndView;
import org.springframework.web.servlet.mvc.support.RedirectAttributes;
import com.jantabank.entities.TransactionInfo;
import com.jantabank.services.AccountService;

```

```

@Controller
public class WithdrawMoneyController
{
    @Autowired private AccountService accountService;

    @GetMapping("services/withdraw-form")
    public String getWithdrawForm()
    {
        return "services/withdraw/withdraw-money-form";
    }

    @GetMapping("services/withdraw.do")
    public String withdrawMoney(int amount,@SessionAttribute("accountno") int
accountno,RedirectAttributes
redirectAttributes,Model model)
    {
        int accountBalance=accountService.getAmount(accountno);
        if(amount>accountBalance)
        {
            model.addAttribute("msg","You do not have sufficient amount into your
account");
            model.addAttribute("amount",amount);
            return "services/withdraw/withdraw-money-form";
        }
        TransactionInfo transaction=accountService.debitAmount(amount,accountno);
        redirectAttributes.addFlashAttribute("transactionInfo", transaction);
        return "redirect:withdraw-success";
    }

    @GetMapping("services/withdraw-success")
    public ModelAndView
withdrawMoneySuccess(@ModelAttribute("transactionInfo") TransactionInfo
transaction)
    {
        ModelAndView modelAndView=new
ModelAndView("services/withdraw/withdraw-money-success");
    }
}

```

```

        modelAndView.addObject("transaction", transaction);
        return modelAndView;
    } }

```

ENTITIES :

ACCOUNT

```

package com.jantabank.entities;
import jakarta.persistence.*;
@Entity
public class Account
{
    @Id
    @GeneratedValue(generator = "account_seq")
    @SequenceGenerator(name = "account_seq",initialValue =
139200010,allocationSize = 1)
    private int accountno;
    private String userid;
    private int amount;
    public int getAccountno() {
        return accountno;
    }
    public void setAccountno(int accountno) {
        this.accountno = accountno;
    }
    public String getUserid() {
        return userid;
    }
    public void setUserid(String userid) {
        this.userid = userid;
    }
    public int getAmount() {
        return amount;
    }
}

```

```

    public void setAmount(int amount) {
        this.amount = amount;
    }

}

```

TRANSACTION INFO

```

package com.jantabank.entities;
import jakarta.persistence.*;

```

```

@Entity
@Table(name = "transaction")
public class TransactionInfo
{
    @Id
    @GeneratedValue(generator = "t_sequence")
    @SequenceGenerator(name = "t_sequence",initialValue = 123456,allocationSize =
1)
    private int tid;
    private int fromaccount;
    private int amount;
    private String type;
    private String date;
    private String time;
    private int toaccount;
    public int getTid() {
        return tid;
    }
    public void setTid(int tid) {
        this.tid = tid;
    }
    public int getFromaccount() {
        return fromaccount;
    }
}

```

```

public void setFromaccount(int fromaccount) {
    this.fromaccount = fromaccount;
}
public int getAmount() {
    return amount;
}
public void setAmount(int amount) {
    this.amount = amount;
}
public String getType() {
    return type;
}
public void setType(String type) {
    this.type = type;
}
public String getDate() {
    return date;
}

public void setDate(String date) {
    this.date = date;
}
public String getTime() {
    return time;
}
public void setTime(String time) {
    this.time = time;
}
public int getToaccount() {
    return toaccount;
}
public void setToaccount(int toaccount) {
    this.toaccount = toaccount;
}

```

```
}  
}
```

USER

```
package com.jantabank.entities;
```

```
import jakarta.persistence.*;
```

```
@Entity
```

```
public class User
```

```
{
```

```
    @Id
```

```
    @Column(length = 50)
```

```
    private String userid;
```

```
    private String password;
```

```
    private String firstname;
```

```
    private String lastname;
```

```
    private String phone;
```

```
    private String email;
```

```
    private String address;
```

```
    private String dob;
```

```
    public String getUserid() {
```

```
        return userid;
```

```
    }
```

```
    public void setUserid(String userid) {
```

```
        this.userid = userid;
```

```
    }
```

```
    public String getPassword() {
```

```
        return password;
```

```
    }
```

```
    public void setPassword(String password) {
```

```
        this.password = password;
```

```
    }
```



```
public String getFirstname() {  
    return firstname;  
}  
public void setFirstname(String firstname) {  
    this.firstname = firstname;  
}  
public String getLastname() {  
    return lastname;  
}  
public void setLastname(String lastname) {  
    this.lastname = lastname;  
}  
public String getPhone() {  
    return phone;  
}  
  
public void setPhone(String phone) {  
    this.phone = phone;  
}  
public String getEmail() {  
    return email;  
}  
public void setEmail(String email) {  
    this.email = email;  
}  
public String getAddress() {  
    return address;  
}  
public void setAddress(String address) {  
    this.address = address;  
}  
public String getDob() {  
    return dob;  
}
```

```

    public void setDob(String dob) {
        this.dob = dob;
    }
}

```

SERVICES :

ACCOUNT SERVICE

INTERFACE

```

package com.jantabank.services;
import com.jantabank.entities.Account;
import com.jantabank.entities.TransactionInfo;

public interface AccountService
{
    Account getAccountByUserId(String userid);
    int getAmount(int accountno);
    TransactionInfo creditAmount(int amount, int accountno);
    TransactionInfo debitAmount(int amount, int accountno);
    String getAccountHolder(int accountno);
    TransactionInfo transferMoney(int amount, int saccountno,int raccountno);
}

```

IMPLEMENTATION CLASS

```

package com.jantabank.services.impl;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.stereotype.Service;
import com.jantabank.entities.Account;
import com.jantabank.entities.TransactionInfo;
import com.jantabank.repositories.AccountRepository;
import com.jantabank.repositories.TransactionRepository;
import com.jantabank.services.AccountService;
import com.jantabank.utility.DateTimeUtility;

```

```

@Service
public class AccountServiceImpl implements AccountService
{
    @Autowired private AccountRepository accountRepository;
    @Autowired private DateTimeUtility dateTimeUtility;
    @Autowired private TransactionRepository transactionRepository;

    public Account getAccountByUserId(String userid)
    {
        return accountRepository.findById(userid);
    }

    public int getAmount(int accountno)
    {
        Account account=accountRepository.findById(accountno).orElse(null);
        return account.getAmount();
    }

    public TransactionInfo creditAmount(int amount, int accountno)

    {
        //Updating amount into account object
        accountRepository.updateAmount(amount,accountno);
        //Persisting transactioninfo object
        TransactionInfo transaction=new TransactionInfo();
        transaction.setFromaccount(accountno);
        transaction.setAmount(amount);
        transaction.setType("Credit");
        transaction.setDate(dateTimeUtility.getCurrentDate());
        transaction.setTime(dateTimeUtility.getCurrentTime());
        transaction.setToaccount(accountno);
        transactionRepository.save(transaction);
    }
}

```

```

        return transaction;
    }

    public TransactionInfo debitAmount(int amount, int accountno)
    {
        accountRepository.updateAmount(-amount,accountno);
        TransactionInfo transaction=new TransactionInfo();
        transaction.setFromaccount(accountno);
        transaction.setAmount(amount);
        transaction.setType("Debit");
        transaction.setDate(dateTimeUtility.getCurrentDate());
        transaction.setTime(dateTimeUtility.getCurrentTime());
        transaction.setToaccount(accountno);
        transactionRepository.save(transaction);
        return transaction;
    }

    public String getAccountHolder(int accountno)
    {
        String name=accountRepository.getNameByAccountno(accountno);
        return name;
    }

    public TransactionInfo transferMoney(int amount, int saccountno, int raccountno)
    {
        //Updating sender amount
        accountRepository.updateAmount(-amount,saccountno);
        //Updating receiver amount
        accountRepository.updateAmount(amount,raccountno);
        saveRecieverTransferTransaction(saccountno,amount,raccountno);
        return saveSenderTransferTransaction(saccountno,amount,raccountno);
    }

    private void saveRecieverTransferTransaction(int saccountno, int amount, int
raccountno)
    {

```

```

        TransactionInfo transaction=new TransactionInfo();
        transaction.setFromaccount(saccountno);
        transaction.setAmount(amount);
        transaction.setType("Credit");
        transaction.setDate(dateTimeUtility.getCurrentDate());

        transaction.setTime(dateTimeUtility.getCurrentTime());
        transaction.setToaccount(raccountno);
        transactionRepository.save(transaction);
    }

    private TransactionInfo saveSenderTransferTransaction(int saccountno,int amount,
int raccountno)
    {
        TransactionInfo transaction=new TransactionInfo();
        transaction.setFromaccount(saccountno);
        transaction.setAmount(amount);
        transaction.setType("Debit");
        transaction.setDate(dateTimeUtility.getCurrentDate());
        transaction.setTime(dateTimeUtility.getCurrentTime());
        transaction.setToaccount(raccountno);
        transactionRepository.save(transaction);
        return transaction;
    }
}

```

USER SERVICE :

INTERFACE

```
package com.jantabank.services;
import com.jantabank.entities.User;

public interface UserService
{
    int saveUserAndCreateAccount(User user);
    User getUser(String userid);
}
```

IMPLEMENTATION CLASS

```
package com.jantabank.services.impl;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.stereotype.Service;
import com.jantabank.entities.Account;
import com.jantabank.entities.User;
import com.jantabank.repositories.AccountRepository;
import com.jantabank.repositories.UserRepository;
import com.jantabank.services.UserService;

@Service
public class UserServiceImpl implements UserService
{
    @Autowired private UserRepository userRepository;
    @Autowired private AccountRepository accountRepository;

    public int saveUserAndCreateAccount(User user)
    {
        //Saving user object
        userRepository.save(user);
        //Saving account object
    }
}
```

```

        Account account=new Account();
        account.setUserid(user.getUserid());
        accountRepository.save(account);
        //Returning account no
        return account.getAccountno();
    }

    public User getUser(String userid)
    {
        return userRepository.findById(userid).orElse(null);
    }
}

```

REPOSITORY:

ACCOUNT REPOSITORY

```

package com.jantabank.repositories;
import org.springframework.data.jpa.repository.JpaRepository;
import org.springframework.data.jpa.repository.Modifying;
import org.springframework.data.jpa.repository.Query;
import org.springframework.data.repository.query.Param;
import com.jantabank.entities.Account;
import jakarta.transaction.Transactional;

public interface AccountRepository extends JpaRepository<Account,Integer>
{
    Account findByUserid(String userid);

    @Modifying
    @Transactional
    @Query("update Account set amount=amount+:arg1 where accountno=:arg2")
    void updateAmount(@Param("arg1") int amount,@Param("arg2") int accountno);

    @Query("select concat(firstname,' ',lastname) from Account a join User u on

```

a.userid=u.userid where

a.accountno=:arg")

```
String getNameByAccountno(@Param("arg") int accountno);  
}
```

TRANSACTION REPOSITORY

```
package com.jantabank.repositories;  
import java.util.List;  
import org.springframework.data.jpa.repository.JpaRepository;  
import org.springframework.data.jpa.repository.Query;  
import org.springframework.data.repository.query.Param;  
import com.jantabank.entities.TransactionInfo;  
public interface TransactionRepository extends  
JpaRepository<TransactionInfo,Integer>  
{  
    @Query("from TransactionInfo where (fromaccount=:arg and toaccount=:arg) or  
(fromaccount=:arg and  
type='Debit') or (toaccount=:arg and type='Credit')")  
    List<TransactionInfo> getTransactionListOfCurrentUser(@Param("arg") int  
accountno);  
}
```

USER REPOSITORY

```
package com.jantabank.repositories;  
import org.springframework.data.jpa.repository.JpaRepository;  
import com.jantabank.entities.User;  
public interface UserRepository extends JpaRepository<User,String>  
{  
}
```


VIEW OF PROJECT

HOME PAGE :

```
<html xmlns:th="http://www.thymeleaf.org">
  <div th:insert=~{reusable-code/header}"></div>
<body>
  <div class="nav">
    <a href="/janta-bank">Home</a>
    <a href="#">About Us</a>
    <div class="dropdown">
      <button class="bt dropdown-toggle" type="button"
id="dropdownMenuButton1" data-bs
toggle="dropdown" aria-expanded="false">
        Account Services
      </button>
      <ul class="ddm dropdown-menu" aria-labelledby="dropdownMenuButton1">
        <li><a class="dropdown-item"
href="/janta-bank/services/view-balance">Show Balance</a></li>
        <li><a class="dropdown-item"
href="/janta-bank/services/deposit-form">Deposit Money</a></li>
        <li><a class="dropdown-item"
href="/janta-bank/services/withdraw-form">Withdraw Money</a></li>
        <li><a class="dropdown-item"
href="/janta-bank/services/transfer-form">Transfer Money</a></li>
        <li><a class="dropdown-item"
href="/janta-bank/services/view-statements">View
Statements</a></li>
      </ul>
    </div>
    <a href="#">Contact</a>
  </div>
  <div class="hero bg-image">
    <h2>Your Trusted Financial Partner</h2>
```

```

</div>
<div th:insert=~{reusable-code/footer}"></div>
</body>
</html>

```

BALANCE PAGE :

SHOW BALANCE PAGE

```

<html xmlns:th="http://www.thymeleaf.org">
<div th:insert=~{@{reusable-code/header}}"></div>
<body class="bg-light text-center">
<div class='bg-image'>
<div class="container py-5">
<div class="card mx-auto shadow" style="max-width:40%;">
<div class="card-body">
<p class="fs-3 card-title">Your Account Balance</p>
<p class="card-text display-6 text-success">₹8377;<span id="balance"
th:text="${amount}"></span></p>
<button type="button" class="btn btn-primary'
onclick="history.back()">Back Home</button>
</div>
</div>
</div>
</div>
<div th:insert=~{reusable-code/footer}"></div>
</body>
</html>

```

SHOW STATEMENT PAGE :

```
<html xmlns:th="http://www.thymeleaf.org">
  <div th:insert="~{@{reusable-code/header}}"></div><style>
    th,td{
      text-align: left;
    }
    .scrollit {
      overflow:scroll;
      max-height:45vh;
    }
    .scrollit thead th {
      position: sticky;
      background:teal;
      color:white;
      top: 0;
    }
  </style>
  <body class="bg-light text-center">
    <div class="bg-image">
      <div class="container py-1">
        <div class="card mx-auto shadow" style="max-width:100%;">
          <div class="card-body" style="height:23vw">
            <div class="table scrollit">
              <table class="table table-bordered">
                <thead>
                  <tr>
                    <th>S.No</th><th>From Account</th><th>To Account</th>
                    <th>Amount</th><th>Date</th><th>Time</th><th>Type</th>
                  </tr>
                </thead>
                <tbody>
                  <tr th:each="tr, iterStat : ${statementList}">
```

```

        <td th:text="\${iterStat.index+1}"></td>
        <td th:text="\${tr.fromaccount}"></td>
        <td th:text="\${tr.toaccount==tr.fromaccount?"Self":tr.toaccount}"></td>
        <td th:class="\${tr.type=="Credit"? "text-success": "text-danger"}'
th:text="\{'&#8377;' + tr.amount}"></td>
        <td th:text="\${tr.date}"></td>
        <td th:text="\${tr.time}"></td>
        <td th:class="\${tr.type=="Credit"? "text-success": "text-danger"}'
th:text="\${tr.type}"></td>
    </tr>
</tbody>
</table>
</div>
</div>
</div>
</div>
<div>
    <a href="/janta-bank" class='btn btn-primary'>Back Home</a>
    <a href="/janta-bank" class='btn btn-secondary'>Download Receipt</a>
</div>
</div>
<div th:insert="\~{reusable-code/footer}"></div>
</body>
</html>

```

DEPOSITE PAGE:

DEPOSIT MONEY PAGE

```
<html xmlns:th="http://www.thymeleaf.org">
  <div th:insert=~{@{reusable-code/header}}"></div>
<body class="bg-light text-center">
  <div class='bg-image'>
    <div class="container py-5">
      <div class="card mx-auto shadow" style="max-width:40%;">
        <div class="card-body py-3">
          <form action="deposit.do">
            <p class='fs-3 py-0 text-primary'>Deposit Form</p>
            <p><input type="number" th:value="${amount}" class="form-control
text-center" name="amount"
placeholder="Enter Amount"></p>
              <button class='btn btn-primary'>Submit</button>
              <button type="button" class='btn btn-secondary'
onclick="history.back()">Back Home</button>
            <p class='my-2'><span class='text-success' th:text="${msg}"></span></p>
          </form>
        </div>
      </div>
    </div>
  </div>
</div>
  <div th:insert=~{@{reusable-code/footer}}"></div>
</body>
</html>
```

DEPOSIT MONEY SUCCESS PAGE

```
<html xmlns:th="http://www.thymeleaf.org">
  <div th:insert="~{@{reusable-code/header}} "></div>
<style>
  th,td{
    text-align: left;
  }
</style>
<body class="bg-light text-center">
  <div class="bg-image">
    <div class="container py-1">
      <div>
        <label class='fs-3 text-white text-decoration-underline'>Deposit Money
Success</label>
      </div>
      <div class="card mx-auto shadow" style="max-width:50%;">
        <div class="card-body">
          <div class="table">
            <table class="table table-bordered">
              <tr>
                <th>Transaction Id</th>
                <td th:text="{transaction.tid}"></td>
              </tr>
              <tr>
                <th>From Account</th>
                <td th:text="{transaction.fromaccount}"></td>
              </tr>
              <tr>
                <th>To Account</th>
                <td>Self</td>
              </tr>
              <tr>
                <th>Account holder</th>
                <td th:text="{session.username}"></td>
            </table>
          </div>
        </div>
      </div>
    </div>
  </div>
</body>
</html>
```

```

        </tr>
        <tr>
            <th>Amount</th>
            <td class="text-success"
th:text="'$'#{&#8377;'+transaction.amount}'"></td>
        </tr>
        <tr>
            <th>Date and Time</th>
            <td><span th:text="'${transaction.date}'"></span><span
style='margin-left:4px'
th:text="'${transaction.time}'"></span></td>
        </tr>
        <tr>
            <th>Transaction Type</th>
            <td class="text-success" th:text="'${transaction.type}'"></td>
        </tr>
    </table>
</div>
</div>
</div>
</div>
<div>
    <a href="/janta-bank" class='btn btn-primary'>Back Home</a>
    <a href="/janta-bank" class='btn btn-secondary'>Download Receipt</a>
</div>
</div>
<div th:insert=~{reusable-code/footer}'"></div>
</body>
</html>

```

TRANSFER MONEY PAGE :

TRANSFER MONEY FORM1

```
<html xmlns:th="http://www.thymeleaf.org">
  <div th:insert="~{@{reusable-code/header}}"></div>
<body class="bg-light text-center">
  <div class="bg-image">
    <div class="container py-5">
      <div class="card mx-auto shadow ch" style='max-width: 40%>
        <div class="card-body py-4">
          <form action="verify-account.do">
            <p class='fs-3 py-0 text-primary'>Transfer Form</p>
            <p>
              <input type="text" th:value="${session.raccountno}"
class="form-control text-center"
name="accountno" placeholder="Enter Account Number">
            </p>
            <button class="btn btn-primary">Submit</button>
            <button type="button" class="btn btn-secondary"
onclick="history.back()">Back Home</button>
            <p class='my-1'>
              <span class='text-danger' th:text="${msg}"></span>
            </p>
          </form>
        </div>
      </div>
    </div>
  </div>
  <div th:insert="~{reusable-code/footer}"></div>
</body>
</html>
```


TRANSFER MONEY FORM2

```
<html xmlns:th="http://www.thymeleaf.org">
  <div th:insert="~{@{reusable-code/header}} "></div>
<body class="bg-light text-center">
  <div class="bg-image">
    <div class="container py-5">
      <div class="card mx-auto shadow ch" style='max-width: 40%>
        <div class="card-body py-0">
          <form action="transfer.do">
            <p class='fs-3 py-0 text-primary'>Transfer Form(Enter amount)</p>
            <div class="row fs-6">
              <div class="col-6 text-start text-dark"><b>Account Holder</b></div>
              <div class="col-6 text-end text-danger" th:text="${session.rname}"></div>
            </div>
            <div class="row fs-6 py-2">
              <div class="col-6 text-start"><b>Account Number</b></div>
              <div class="col-6 text-end text-danger"
th:text="${session.raccountno}"></div>
            </div>
            <p><input type="text" th:value="${amount}" class="form-control text-center"
name="amount"
placeholder="Enter amount"></p>
            <button class="btn btn-primary">Submit</button>
            <button type="button" class="btn btn-secondary"
onclick="history.back()">Back Home</button>
            <p class='my-1'><span class='text-danger' th:text="${msg}"></span></p>
          </form>
        </div>
      </div>
    </div>
  </div>
  <div th:insert="~{@{reusable-code/footer}} "></div>
</body>
</html>
```

TRANSFER MONEY SUCCESS PAGE

```
<html xmlns:th="http://www.thymeleaf.org">
  <div th:insert="~{@{reusable-code/header}} "></div>
<style>
  th,td{
    text-align: left;
  }
</style>
<body class="bg-light text-center">
  <div class="bg-image">
    <div class="container py-1">
      <div>
        <label class='fs-3 text-white text-decoration-underline'>Transfer Money
Success</label>
      </div>
      <div class="card mx-auto shadow" style="max-width:50%;">
        <div class="card-body">
          <div class="table">
            <table class="table table-bordered">
              <tr>
                <th>Transaction Id</th>
                <td th:text="{transaction.tid}"></td>
              </tr>
              <tr>
                <th>From Account</th>
                <td th:text="{transaction.fromaccount}"></td>
              </tr>
              <tr>
                <th>Sender</th>
                <td th:text="{session.username}"></td>
              </tr>
              <tr>
                <th>To Account</th>
```

```

        <td th:text="{transaction.toaccount}"></td>
    </tr>
    <tr>
        <th>Receiver</th>
        <td th:text="{session.rname}"></td>
    </tr>
    <tr>
        <th>Amount</th>
        <td class="text-danger"
th:text="{ '&#8377;' + transaction.amount}"></td>
    </tr>
    <tr>
        <th>Date and Time</th>
        <td><span th:text="{transaction.date}"></span><span
style='margin-left:4px'
th:text="{transaction.time}"></span></td>
    </tr>
    <tr>
        <th>Transaction Type</th>
        <td class="text-danger">Transfer</td>
    </tr>
</table>
</div>
</div>
</div>
<div>
    <a href="/janta-bank" class="btn btn-primary">Back Home</a>
    <a href="/janta-bank" class="btn btn-secondary">Download Receipt</a>
</div>
</div>
<div th:insert="{~{reusable-code/footer}}"></div>
</body>
</html>

```

WITHDRAW PAGE :

WITHDRAW FORM PAGE

```
<html xmlns:th="http://www.thymeleaf.org">
  <div th:insert="~{@{reusable-code/header}}"></div>
  <body class="bg-light text-center">
    <div class="bg-image">
      <div class="container py-5">
        <div class="card mx-auto shadow ch" style="max-width:40%;">
          <div class="card-body py-5">
            <form action="withdraw.do">
              <p class='fs-3 py-0 text-primary'>Withdraw Form</p>
              <p><input type="number" th:value="${amount}" class="form-control
text-center" name="amount"
placeholder="Enter Amount"></p>
              <button class='btn btn-primary'>Submit</button>
              <button type="button" class='btn btn-secondary'
onclick="history.back()">Back Home</button>
              <p class='my-2'><span class='text-danger' th:text="${msg}"></span></p>
            </form>
          </div>
        </div>
      </div>
    </div>
  </div>
  <div th:insert="~{@{reusable-code/footer}}"></div>
</body>
</html>
```

WITHDRAW MONEY SUCCESS PAGE

```
<html xmlns:th="http://www.thymeleaf.org">
  <div th:insert="~{@{reusable-code/header}}"></div>
  <style>
    th,td{
      text-align: left;
```

```

}
</style>
<body class="bg-light text-center">
  <div class='bg-image'>
    <div class="container py-1">
      <div>
        <label class='fs-3 text-white text-decoration-underline'>Withdraw Money
Success</label>
      </div>
      <div class="card mx-auto shadow" style="max-width:50%;">
        <div class="card-body">
          <div class="table">
            <table class="table table-bordered">
              <tr>
                <th>Transaction Id</th>
                <td th:text="\${transaction.tid}"></td>
              </tr>
              <tr>
                <th>From Account</th>
                <td th:text="\${transaction.fromaccount}"></td>
              </tr>
              <tr>
                <th>To Account</th>
                <td>Self</td>
              </tr>
              <tr>
                <th>Account holder</th>
                <td th:text="\${session.username}"></td>
              </tr>
              <tr>
                <th>Amount</th>
                <td class="text-danger"
th:text="\${'&#8377;'+transaction.amount}"></td>
              </tr>

```

```

        <tr>
            <th>Date and Time</th>
            <td><span th:text="{transaction.date}"></span><span
style='margin-left:4px'
th:text="{transaction.time}"></span></td>
        </tr>
        <tr>
            <th>Transaction Type</th>
            <td class="text-danger" th:text="{transaction.type}"></td>
        </tr>
    </table>
</div>
</div>
</div>
</div>
<div>
    <a href="/janta-bank" class="btn btn-primary">Back Home</a>
    <a href="/janta-bank" class="btn btn-secondary">Download Receipt</a>
</div>
</div>
<div th:insert="~{reusable-code/footer}"></div>
</body>
</html>

```

USER PAGES :

LOGIN PAGE

```
<html xmlns:th="http://www.thymeleaf.org">
  <div th:insert=~{@{user/user-header}}"></div>
  <body>
    <div class="bg bg-image">
      <div class="container d-flex justify-content-center align-items-center py-1">
        <div class="card p-4 shadow-lg" style="width: 40%;height: 70%;">
          <h4 class="text-center mb-4">Login</h4>
          <form action="/janta-bank/user/authenticate-user" method="post">
            <div class="mb-3">
              <label for="username" class="form-label">User Id</label>
              <input type="text" th:value="${uid}" class="form-control" name="userid"
placeholder="Enter user
id" required>
            </div>
            <div class="mb-3">
              <label for="password" class="form-label">Password</label>
              <input type="password" class="form-control" name="password"
placeholder="Enter your password"
required>
            </div>
            <div class="mb-3 my-4">
              <p><button class="btn btn-primary w-100">Login</button></p>
            </div>
          </form>
          <div class="text-center mb-3">
            <a href="/janta-bank/user/registration">Register User?</a>
          </div>
          <div class='text-center'>
            <label class='text-danger' th:text="${msg}"></label>
          </div>
        </div>
      </div>
    </div>
  </div>
```

```

</div>
</div>
<div th:insert=~{@{reusable-code/footer}}"></div>
</body>
</html>

```

LOGOUT PAGE

```

<html xmlns:th="http://www.thymeleaf.org">
  <div th:insert=~{@{user/user-header}}"></div>
  <body class="bg-light text-center">
    <div class="bg-image">
      <div class="container py-5">
        <div class="card mx-auto shadow" style="max-width: 50%;">
          <div class="card-body">
            <p class="fs-2 card-text display-6 text-success"
th:text="${username}"></p>
            <p class="fs-2 card-title text-danger">You have been logged out
successfully</p>
            <a class="btn btn-primary" href="/janta-bank">Click here to login</a>
          </div>
        </div>
      </div>
    </div>
  </div>
  <div th:insert=~{@{reusable-code/footer}}"></div>
</body>
</body>
</html>

```


REGISTRATION PAGE :

REGISTER USER PAGE

```
<html xmlns:th="http://www.thymeleaf.org">
<div th:insert=~{@{user/user-header}}"></div>
<body>
<div class="bg bg-image">
<div class="container d-flex justify-content-center align-items-center py-1">
<div class="card p-3 shadow-lg" style="width: 60%">
<h4 class="text-center mb-4">Registration</h4>
<div class='card-body'>
<form action="register-user" method="post" onsubmit="return
validatePassword()">
<div class='row'>
<div class="col-12">
<label for="userid">User Id</label>
<input type="text" class='form-control' name='userid' id='userid' required>
</div>
</div>
<div class='row my-3'>
<div class="col-6">
<label for="password">Password</label>
<input type="password" class='form-control' onkeyup="validatePassword()"
name='password'
id='password' required>
<span id="message"></span>
</div>
<div class="col-6">
<label for="rpassword">Repeat Password</label>
<input type="password" class='form-control'
onkeyup="validatePasswordMatch()" name='rpassword'
id='rpassword' required>
<span id="matchMessage"></span>
</div>
```

```

</div>
<div class='row my-3'>
  <div class="col-6">
    <label for="firstname">First name</label>
    <input type="text" class='form-control' name='firstname' id='firstname'
required>
  </div>
  <div class="col-6">
    <label for="lastname">Last name</label>
    <input type="text" class='form-control' name='lastname' id='lastname' required>
  </div>
</div>
<div class='row my-3'>
  <div class="col-6">
    <label for="phone">Phone number</label>

    <input type="text" class='form-control' name='phone' id='phone' required>
  </div>
  <div class="col-6">
    <label for="email">Email id</label>
    <input type="text" class='form-control' onkeyup="validateEmail()" name='email'
id='email' required>
    <span id="emailMessage"></span>
  </div>
</div>
<div class='row my-3'>
  <div class="col-6">
    <label for="address">Address</label>
    <input type="text" class='form-control' name='address' id='address' required>
  </div>
  <div class="col-6">
    <label for="dob">Date of Birth</label>
    <input type="date" class='form-control' th:max="{cdate}" name='dob' id='dob'

```

```

required>
    </div>
</div>
<div class='row my-3'>
    <div class="col-12 text-center">
        <button class='btn btn-primary mx-2' style="width:35%" onclick="return
matchPassword()">Register
User</button>
        <input type="button" value='Cancel' class='btn btn-secondary'
style="width:35%"
onclick="history.back()">
    </div>
</div>
</form>
</div>
</div>
</div>
<div th:insert="~{@{reusable-code/footer}}"></div>
</body>
</html>

```

REGISTER USER SUCCESS PAGE :

```

<html xmlns:th="http://www.thymeleaf.org">
<div th:insert="~{@{user/user-header}}"></div>
<head>
    <script>
        document.addEventListener("keydown", function (event) {
            if (event.key === "F5" || (event.ctrlKey && event.key === "r")) {
                event.preventDefault();
            }
        });
    </script>
</head>

```

```

<body class="bg-light text-center" onload="stopRefresh()">

<div class="bg-image">
<div class="container py-5">
  <div class="card mx-auto shadow" style="max-width: 50%;">
    <div class="card-body">
      <p class="fs-2 card-text display-6 text-success"
th:text="{username}"></p>
      <p class="fs-2 card-title">You have been registered successfully</p>
      <p class="card-text display-6 text-success">Your account number is <span
id="accountno"
th:text="{accountno}"></span></p>
      <a class="btn btn-primary" href="/janta-bank">Click here to login</a>
    </div>
  </div>
</div>
</div>
<div th:insert="~{@{reusable-code/footer}}"></div>
</body>
</html>

```

USER HEADER PAGE :

```

<html xmlns:th="http://www.thymeleaf.org">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Bank Home Page</title>
  <link th:href="@{/CSS/bootstrap.min.css}" rel="stylesheet">
  <link th:href="@{/CSS/bank.css?v=2.0}" rel="stylesheet">
  <script type="text/javascript" th:src="@{/JS/bank.js?v=2.0}"></script>
</head>
<div class="header">
  <div class="row">
    <div class="col-6">

```

```

        <p class='fs-3'>Welcome to Janta Bank</p>
    </div>
    <div class="col-6">
        <h2>Your Trusted Financial Partner</h2>
    </div>
</div>
</div>

```

REUSABLE CODE :

HEADER

```

<html xmlns:th="http://www.thymeleaf.org">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Bank Home Page</title>
    <link th:href="@{/CSS/bootstrap.min.css}" rel="stylesheet">
    <link th:href="@{/CSS/bank.css?v=2.0}" rel="stylesheet">
    <script type="text/javascript"
th:src="@{/JS/bootstrap.bundle.min.js?v=2.0}"></script>
</head>
<div class="header">
    <div class="row">
        <div class='col-3'>
            <span class='fs-5 text-info'>Savings Account</span><br> <span
            class='fs-6' th:text="${session.accountno}"></span>
        </div>
        <div class='col-3'>
            <p class='fs-3'>Janta Bank</p>
        </div>
        <div class='col-3'>
            <span class='fs-5 text-info'>Your Name</span><br> <span
            class='fs-6' th:text="${session.username}"></span>
        </div>
    </div>

```

```
<div class='col-3'>
  <a href="/janta-bank/user/logout" class='fs-5 text-info'>Logout</a>
</div>
</div>
</div>
```

FOOTER :

```
<div class="services">
  <h2>Our Services</h2>
  <div>
    <h3>Online Banking</h3>
    <p>Access your account anytime, anywhere.</p>
  </div>
  <div>
    <h3>Loans</h3>
    <p>Flexible loan options tailored for you.</p>
  </div>
  <div>
    <h3>Investments</h3>
    <p>Grow your wealth with our expert advice.</p>
  </div>
</div>
<div class="footer">
  <p>&copy; 2025 Our Bank. All rights reserved.</p>
</div>
```

DATABASE CONECTIVITY FILE

APPLICATION PROPERTIES FILE :

```
spring.application.name=jantabank-thymeleaf-datajpa
spring.datasource.driver-class-name=com.mysql.cj.jdbc.Driver
spring.datasource.url=jdbc:mysql://localhost:3306/boot10?createDatabaseIfNotExist=
true
spring.datasource.username=root
spring.datasource.password=mysql
#configuring hibernate properties
spring.jpa.hibernate.ddl-auto=update
spring.jpa.show-sql=true
spring.jpa.properties.hibernate.dialect=org.hibernate.dialect.MySQL8Dialect
server.servlet.context-path=/janta-bank
spring.jpa.open-in-view=false
```

CHAPTER NO. 6

TESTING

6. TESTING

6.1 TESTING

Testing is a critical phase in the software development life cycle that ensures the functionality, reliability, and performance of the *Bank Management System*. Throughout the development process, multiple levels of testing were performed to validate the behavior of the application against its design and user expectations. These levels included unit testing, integration testing, system testing, and user acceptance testing (UAT) to ensure that each module, as well as the complete system, functioned accurately and securely in a real-time environment.

Unit testing focused on verifying individual modules such as user login, balance enquiry, deposit, withdrawal, and fund transfer. These tests helped ensure that each service method and controller performed correctly in isolation, handling both valid inputs and edge cases gracefully. Integration testing followed, where the interactions between modules—such as between user accounts and transaction processing—were validated to ensure smooth data flow and coordination across different components of the system.

System testing was conducted to evaluate the entire application's performance, usability, and compatibility. Functional test cases were designed to confirm features like PIN-based authentication, role-based redirection (user/admin), real-time balance updates, session timeout handling, and error reporting. Tests were executed manually and through automation to detect bugs and inconsistencies under different usage scenarios. Validation testing ensured the application fulfilled all specified user requirements, while verification confirmed adherence to design documents and coding standards.

Regression testing was also performed after each code update or module enhancement to ensure existing features remained unaffected. Additionally, debugging tools and detailed log tracking were used throughout to identify system issues and streamline the bug-fixing process.

The testing phase concluded with User Acceptance Testing (UAT), where real users interacted with the system in a controlled environment. Their feedback was

instrumental in fine-tuning usability, fixing overlooked issues, and ensuring readiness for real-world deployment. Overall, this comprehensive testing process helped deliver a secure, reliable, and user-centric banking platform.

6.2 TESTING CASES

Several functional and non-functional testing cases were designed to validate the key modules and workflows of the Bank Management System. Each test case was structured to verify specific features like user registration, login, deposits, withdrawals, fund transfers, admin controls, and session management. These test cases were crucial in identifying system errors, verifying data flow accuracy, and ensuring a secure and seamless user experience for both administrators and customers.

Functional testing scenarios included actions such as creating new user accounts, logging in with valid and invalid credentials, performing transactions, checking balance updates, and generating transaction history. **Non-functional testing** focused on performance, security, session timeout, error handling, and system behavior under concurrent usage. For instance, one test case verified whether a user could successfully transfer funds and receive confirmation, while another validated admin access controls when approving or rejecting suspicious transactions.

Boundary value analysis was used to test input fields like password length, maximum deposit limits, and invalid account numbers. Error-handling cases ensured that invalid logins, incorrect PIN entries, or insufficient funds were addressed with clear and user-friendly messages. Test scenarios were also created to simulate concurrent sessions by multiple users and admins to validate session handling and prevent unauthorized access.

Each test case included a description of the scenario, input data, expected output, and actual result. This structured approach allowed for effective debugging and ensured that every function worked as intended. **Verification** confirmed that all modules followed the defined system design, while **validation** ensured they met user expectations. **Regression testing** was performed after updates to check that existing features remained unaffected.

Login Module Test Cases:

- Valid and invalid login credentials
- Role-based access control (User vs Admin dashboards)

- Session timeout and auto logout after inactivity

User Registration Test Cases:

- Registering with unique vs duplicate email/PIN
- Field validation (mandatory fields, email format)
- Edge cases (short/long password, empty fields)

Transaction Module Test Cases:

- Depositing valid/invalid amounts
- Withdrawing with insufficient balance
- Real-time balance update after transaction
- Generating and viewing transaction history

Fund Transfer Test Cases:

- Transferring to valid vs invalid account numbers
- OTP validation success/failure
- Fund deduction from sender and addition to receiver
- Transfer status: pending, completed, failed

Admin Module Test Cases:

- Admin login and access restrictions
- Viewing and managing user accounts
- Approving/deactivating suspicious accounts
- Monitoring transaction logs and reports

Each of these test cases contributed to delivering a reliable, bug-free, and secure banking application capable of functioning smoothly in real-world scenarios.

6.3 VERIFICATION AND VALIDATION

The Bank Management System underwent both verification and validation processes to ensure the application met technical specifications and fulfilled real-world banking requirements. These quality assurance practices were conducted throughout the development life cycle to confirm the system's correctness, consistency, and usability from both developer and user perspectives.

Verification involved the use of static and dynamic testing methods to ensure that the application was being built correctly according to predefined requirements and system design documents. Code reviews, walkthroughs, and requirement traceability matrix (RTM) checks were performed to match functionalities with documented use cases. Each core module—such as user registration, login with PIN, transaction processing, fund transfer, and admin management—was verified to ensure accurate logic implementation. For example, during verification, it was confirmed that account balances updated correctly after each deposit or withdrawal, fund transfers executed only with valid credentials, and only authenticated admins could access sensitive user data and transaction logs. Static verification activities like syntax checking and code analysis were complemented by dynamic testing such as unit tests and integration tests to validate the internal consistency of each module.

Validation was carried out to ensure that the system functioned effectively in real-world scenarios and satisfied the needs of its end users, including customers and administrators. This involved executing simulated banking operations like user onboarding, initiating transactions, handling incorrect PIN entries, and observing how the system responded under normal and edge case conditions. For example, scenarios were tested where a user attempted to transfer funds to an invalid account, or when two users logged in concurrently to access and modify their accounts. Feedback was collected from test users to assess usability, response time, and error messaging. Additionally, administrative functions such as account approval, report generation, and system monitoring were validated to reflect actual banking workflows. The system's ability to handle expired sessions, unauthorized access attempts, and simultaneous fund transfers was also evaluated to confirm its stability and security.

6.4 DEBUGGING

Debugging is an essential process in software development that involves identifying, analyzing, and resolving errors or defects in the code to ensure smooth and reliable system functionality. In the *Bank Management System*, debugging was carried out continuously throughout the development and testing phases to ensure each module operated according to specifications and responded accurately to user inputs. The debugging process was supported by a combination of manual inspection, structured testing, log analysis, and the use of integrated development environment (IDE) tools like IntelliJ IDEA and Eclipse.

Most bugs identified were related to logical errors, data mismatch, incorrect flow of control, invalid input handling, and UI inconsistencies. For example, during the transaction module development, one issue involved incorrect balance deduction during fund transfer due to the transaction order in the service logic. Similarly, in the login module, several attempts were required to properly synchronize session management and timeouts. These bugs were traced and resolved using step-by-step debugging tools and console log outputs to monitor the behavior of variables and functions at runtime.

To streamline the debugging process, server logs and exception handlers were configured to capture runtime errors and unexpected conditions. In Spring Boot, structured logs using built-in logging frameworks helped identify the source of exceptions like `NullPointerException`, `SQLException`, or invalid user sessions. Breakpoints were frequently used to pause program execution and inspect variable values and control flows during test scenarios.

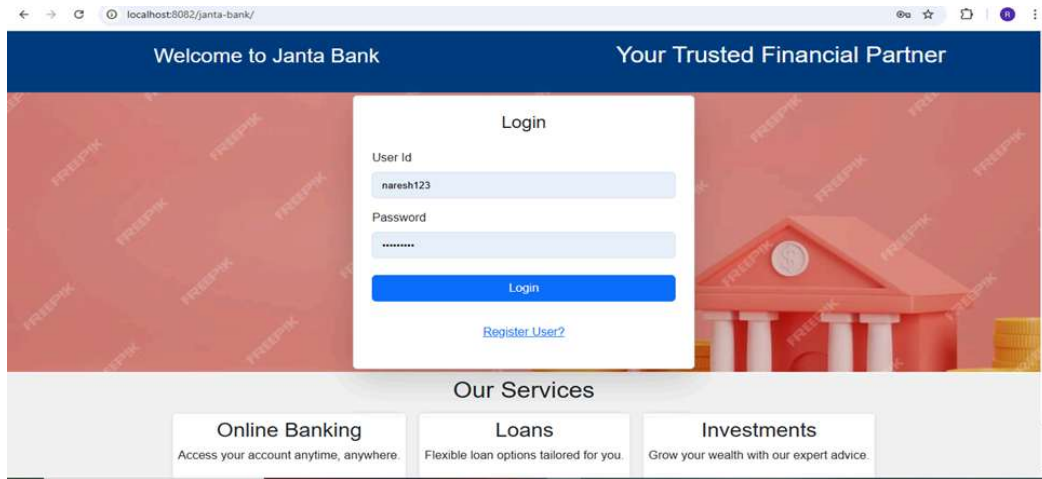
Debugging was not limited to the backend; frontend validation and browser console tools were also utilized to trace rendering errors, incorrect form submissions, or missing field validations. JavaScript console logs and network request inspectors in browsers like Chrome helped identify UI-level bugs during user registration, login, and transaction form submissions.

Each issue identified during testing—whether functional, visual, or logical—was logged, assigned a priority level, and resolved with follow-up regression testing to ensure that the fix did not break existing functionalities.

CHAPTER NO. 7

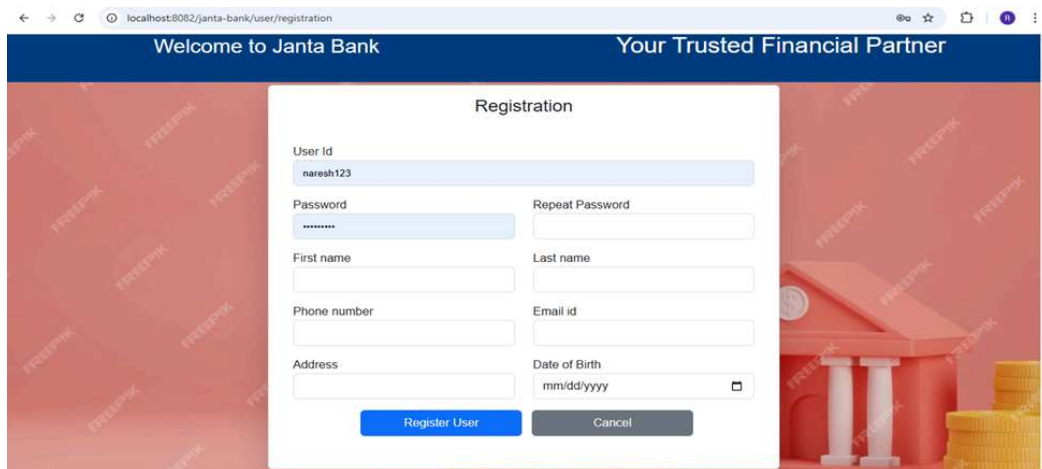
SCREENSHOTS

7. SCREENSHOTS



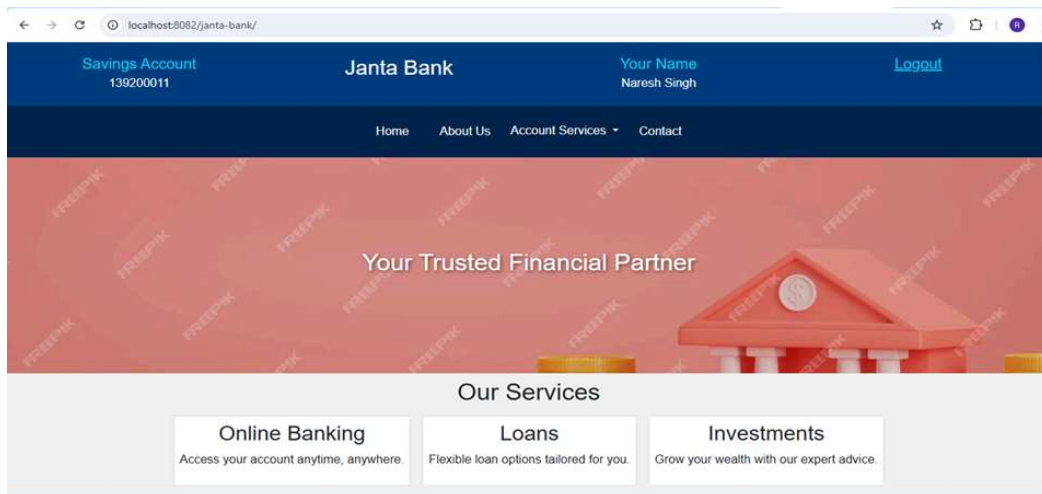
The screenshot shows the login page of Janta Bank. The header features the text "Welcome to Janta Bank" and "Your Trusted Financial Partner". The main content area has a red background with a faint watermark of the word "FREEMIX". A central white box contains the "Login" form with fields for "User Id" (containing "naresh123") and "Password" (masked with dots). Below these fields is a blue "Login" button and a link for "Register User2". At the bottom, a section titled "Our Services" includes three cards: "Online Banking" (Access your account anytime, anywhere), "Loans" (Flexible loan options tailored for you), and "Investments" (Grow your wealth with our expert advice).

(Fig. 7.1) Login Page

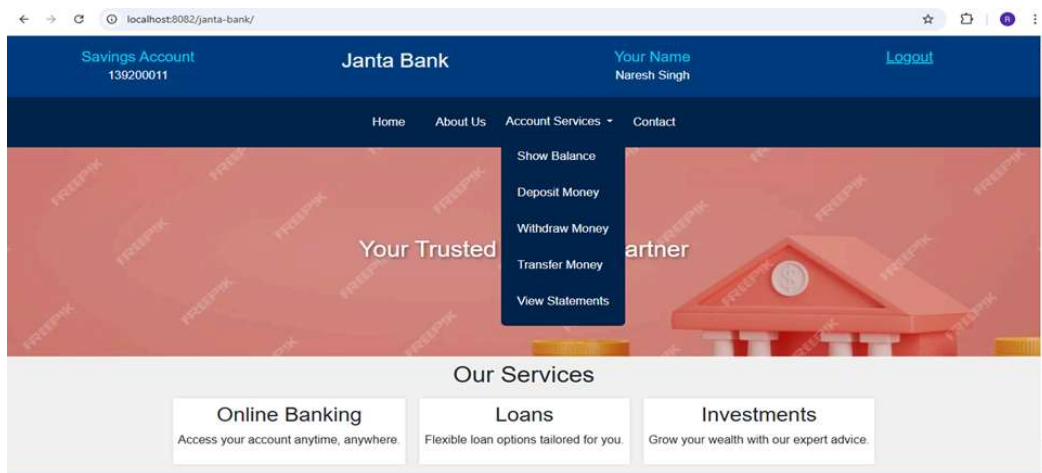


The screenshot shows the registration page of Janta Bank. The header is identical to the login page. The main content area has a red background with a faint watermark of the word "FREEMIX". A central white box contains the "Registration" form. It includes fields for "User Id" (containing "naresh123"), "Password" (masked with dots), and "Repeat Password". Below these are fields for "First name", "Last name", "Phone number", "Email id", "Address", and "Date of Birth" (with a date picker set to "mm/dd/yyyy"). At the bottom of the form are two buttons: "Register User" (blue) and "Cancel" (grey).

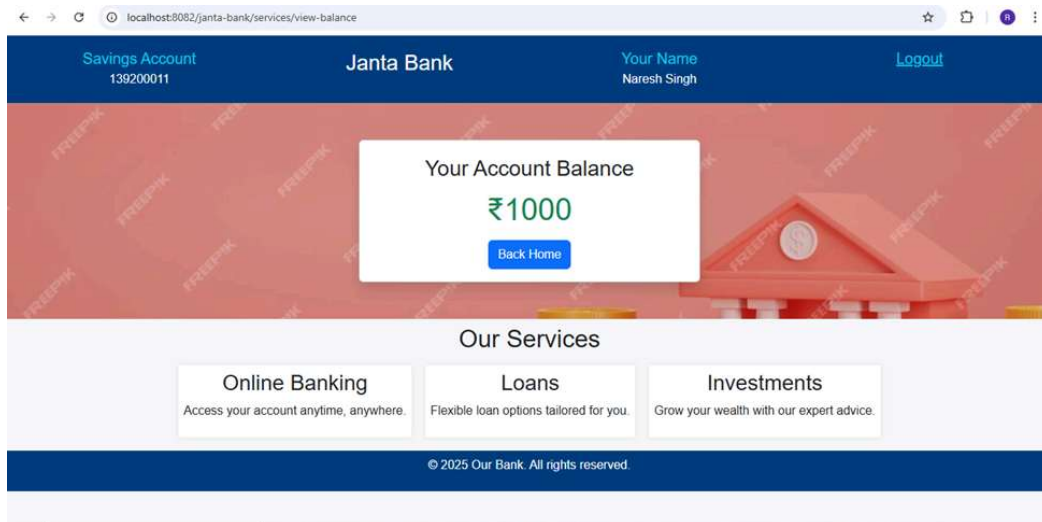
(Fig. 7.2) Registration Page



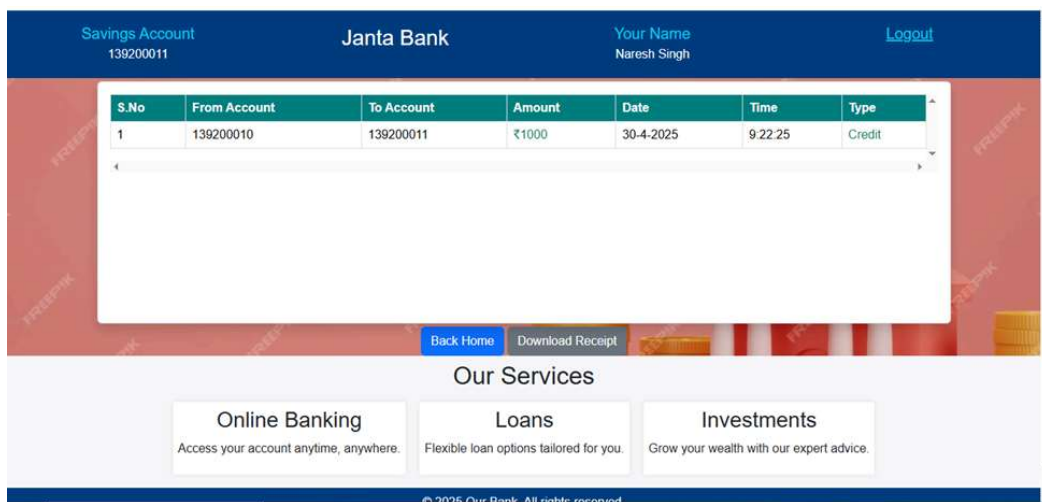
(Fig. 7.3) Partner Landing Page



(Fig. 7.4) Services Offered



(Fig. 7.5) Accounts Balance



(Fig. 7.6) Record Page

CHAPTER NO. 8

IMPLEMENTATION

8. IMPLEMENTATION

Implementation is the phase where the designed system is brought to life through coding, integration, and deployment. In the *Bank Management System* project, implementation involved developing the backend logic, designing a responsive frontend interface, setting up the database, and ensuring that all modules function cohesively. The system was built using Java with the Spring Boot framework for backend development, MySQL as the relational database, and HTML, CSS, Bootstrap, and Thymeleaf for the user interface. The implementation followed the MVC (Model-View-Controller) architecture, ensuring a clear separation of concerns between data processing, user interaction, and control logic.

During implementation, all modules were developed incrementally and tested simultaneously to maintain quality and modularity. The User Module was implemented first, allowing customer registration, PIN-based login, session handling, and secure logout. This was followed by the Account Management Module, where account creation, balance tracking, and transaction linking were developed. The Transaction Module included deposit and withdrawal functionalities with real-time balance updates and error handling for insufficient funds. The Fund Transfer Module enabled intra-bank transfers with validation, OTP confirmation, and transaction status updates. The Admin Module allowed administrators to monitor user activity, approve or deactivate accounts, and view system logs.

Database schema and table relationships were implemented using MySQL, with JPA/Hibernate handling ORM (Object-Relational Mapping). Controllers, services, and repositories were created for each module to streamline functionality and enhance maintainability. Frontend implementation ensured that all pages were responsive, user-friendly, and securely connected with backend endpoints. RESTful endpoints and form-based routing enabled smooth navigation and interaction.

Finally, the system was deployed in a local server environment using Apache Tomcat, and testing was carried out to validate the application's readiness. Each component was integrated into the final build after ensuring its functionality and compatibility. The implementation process followed best coding practices, version control (via Git), and proper documentation to facilitate future maintenance and scalability.

CHAPTER NO. 9

LIMITATION

9. LIMITATION

Despite its robust features and functionality, the *Bank Management System* has certain limitations that must be acknowledged. The current version is designed for small to medium-scale usage and is not optimized for high-concurrency environments, which may affect performance during peak usage. While it successfully automates core banking operations like deposits, withdrawals, and fund transfers, it does not yet support advanced banking services such as loans, credit card management, or investment tracking. The system is built primarily for desktop web browsers and lacks mobile app support, limiting accessibility for users on smartphones. Furthermore, the application currently operates in a single-language (English) environment, which may not be ideal for users from diverse linguistic backgrounds. Security is handled at a basic level with authentication and session control, but advanced features such as AI-based fraud detection or biometric authentication are not integrated. Notifications are only partially implemented, with no real-time SMS or email alerts for all transaction types. Additionally, the system lacks built-in tools for automated backup and disaster recovery, making data protection dependent on manual processes. It also assumes uninterrupted internet connectivity, which may be a barrier in low-bandwidth or rural areas. Lastly, integration with third-party APIs, national banking networks, or payment gateways is not present in this version, limiting its scope for real-world deployment in multi-bank or cross-border environments..

CHAPTER NO. 10
CONCLUSION AND FUTURE SCOPE

10. CONCLUSION AND FUTURE SCOPE

10.1 CONCLUSION

The Bank Management System project represents a successful implementation of core digital banking functionalities through a secure, scalable, and user-friendly web-based application. With the growing need for digitization in the financial sector, this system addresses many of the inefficiencies present in traditional banking environments, such as manual paperwork, long queues, data redundancy, and limited service accessibility. The project was conceptualized, designed, and developed to bring ease of banking operations to both end-users (customers) and administrators, ensuring that essential tasks like account creation, fund transfers, transaction history tracking, and balance inquiry could be completed online, securely, and efficiently.

The system architecture is based on the **Model-View-Controller (MVC)** framework, separating the user interface, business logic, and data access layers. This modular approach made the system more maintainable, testable, and scalable. Technologies such as **Java with Spring Boot, MySQL, HTML/CSS, Bootstrap, and Thymeleaf** were integrated to create a responsive and interactive platform. Each module—including login, registration, deposit, withdrawal, fund transfer, and administrative controls—was carefully designed to meet user expectations and deliver functionality that mirrors real-world banking operations.

Through the application of proper **Software Development Life Cycle (SDLC)** methodologies, the system progressed through clearly defined stages of requirement gathering, design, implementation, testing, and deployment. Functional and non-functional requirements were gathered through analysis of existing manual systems, stakeholder input, and modern banking standards. The detailed system analysis and feasibility study provided a strong foundation for a technically and economically viable solution.

Extensive **testing**, including unit testing, integration testing, system testing, and user acceptance testing (UAT), was carried out to ensure the accuracy, reliability, and usability of the application. Each test case was designed to simulate real-world scenarios such as insufficient balance handling, secure login validation, incorrect account number entry, and concurrent user logins. Debugging tools, console logs, and

exception handling mechanisms were used effectively to identify and resolve issues during the testing and development stages.

The **user interface** was developed with simplicity and usability in mind. It provides seamless navigation for both customers and administrators. The system ensures role-based access, where customers can manage their personal bank accounts, while administrators can control user accounts, monitor transactions, and generate reports. Basic security measures, such as PIN-based login, session tracking, and validation checks, help in safeguarding user data and maintaining transactional integrity.

However, the project is not without limitations. While it performs all basic banking operations reliably, it does not include modules for services like loan management, credit card issuance, investment planning, or customer support integration. The system currently supports only English and is built primarily for desktop browsers, which may limit accessibility in multilingual or mobile-first user bases. Also, while secure, the application does not include advanced features like biometric authentication, AI-based fraud detection, or integration with external banking APIs or payment gateways. These features can be considered for future expansion.

Despite these limitations, the system serves as a robust prototype for digital banking transformation in small to medium-scale financial institutions. It proves that core banking services can be effectively digitized with open-source tools and proper development methodologies. The project offers a high return on investment by reducing operational costs, minimizing human error, improving customer service, and enabling 24/7 access to banking services.

In conclusion, the *Bank Management System* successfully demonstrates the potential of integrating technology into essential financial services. It provides a reliable platform for both users and administrators to perform day-to-day banking operations in a secure, efficient, and accessible manner. With further enhancements, the system has the potential to evolve into a comprehensive digital banking solution adaptable to larger financial institutions and integrated fintech environments. This project not only meets its intended objectives but also serves as a foundation for continued learning, development, and innovation in the field of web-based financial systems.

10.2 FUTURE SCOPE

The Bank Management System lays the groundwork for a comprehensive digital banking platform, but there remains significant potential for future enhancements and expansion. As technology continues to evolve and user expectations grow, this system can be upgraded and adapted to meet broader and more complex financial needs.

One of the most promising areas for future development is the **integration of mobile banking support**. A dedicated mobile application compatible with Android and iOS platforms would enhance accessibility, allowing users to manage their accounts, perform transactions, and receive real-time alerts directly from their smartphones. This would be especially beneficial in a mobile-first economy where users prefer banking on-the-go.

Another key area is the **addition of advanced banking modules** such as loan management, credit card issuance, investment tracking, and recurring payments. Implementing these features would transform the application from a basic banking utility into a complete financial management system. These modules could include EMI calculators, loan application workflows, and interest tracking to meet broader customer needs.

Multilingual support is another vital enhancement, particularly for deployment in linguistically diverse regions. Offering the application in regional languages would improve user engagement and make digital banking more inclusive.

In terms of security, the system can benefit from the **implementation of biometric authentication (fingerprint/face recognition)** and **two-factor authentication (2FA)** for added protection. Additionally, **AI-driven fraud detection systems** can be integrated to monitor unusual transaction patterns and protect against cyber threats.

From a backend perspective, future upgrades could include **integration with government banking APIs, UPI, and payment gateways** to support real-time, third-party payments and cross-platform financial operations. These features would make the application viable for deployment in commercial and cooperative banks at a larger scale.

Cloud deployment and database optimization can be considered to improve

scalability, availability, and performance under heavy traffic conditions. Implementing **automated backup and disaster recovery mechanisms** would further enhance system reliability and business continuity.

Additionally, a **customer support chatbot**, real-time chat services, and feedback modules could be introduced to provide instant assistance and build trust between users and the bank.

In summary, the *Bank Management System* has immense potential for future growth. With the addition of new features, improved security, mobile and multilingual support, and real-time integrations, the system can evolve into a full-fledged digital banking ecosystem ready for real-world deployment in diverse environments. These enhancements would not only improve user experience but also align the system with modern banking standards and innovations.

REFERENCES

- 1. Oracle Java Documentation** – Official Java SE API and Developer Guides. URL: <https://docs.oracle.com/javase/>
- 2. MySQL Official Documentation** – Structured documentation on MySQL architecture, installation, and queries. URL: <https://dev.mysql.com/doc/>
- 3. Spring Framework Guides** – Spring Boot and MVC architecture for web applications. URL: <https://spring.io/guides>
- 4. CETPA Infotech Pvt. Ltd.** – Company profile, training modules, and course content used during industrial training. URL: <https://www.cetpainfotech.com>
- 5. W3Schools** – Java & SQL Tutorials – For syntax references and programming examples. URL: <https://www.w3schools.com>
- 6. GeeksforGeeks** – Java & Database Management – Concepts of Object-Oriented Programming and DBMS. URL: <https://www.geeksforgeeks.org>
- 7. Stack Overflow** – Community-sourced technical discussions and code-level clarifications. URL: <https://stackoverflow.com>
- 8. Github** – Open-source Java Projects – For reference code related to banking systems using MVC. URL: <https://github.com>
- 9. Baeldung** – Spring Boot Examples and Tutorials – Best practices and controller-service patterns. URL: <https://www.baeldung.com>